Master prompt: build the 500,000-session platform

MASTER.01 | V2 MANDATE | COPY THIS FIRST

Emergent, replace the previous infrastructure envelope with a system designed and tested for 500,000 students simultaneously taking timed examinations. Treat this as active stateful exam traffic, with answer commitments, deadlines and submission bursts, rather than an account-count or idle-browser target.

Build the complete system

Implement the blue student and administrator product from Volumes 1-4 using the immutable source, package, API, cell, security and operating contracts in this volume. Retain the verified exam-specific rules and the 60-analysis definitions. This revision supersedes previous 10,000-user launch and 50,000-user stretch infrastructure limits as the final design target; those may remain intermediate test stages. Every claimed operating envelope must carry dated test evidence.

Deliver a portable frontend and backend repository, database migrations, infrastructure-as-code, typed APIs, source vault, server secrets service, package compiler, cell-aware attempt service, exact scorer, asynchronous analytics, release controls and operator tools. Separate the source vault from secret storage and from student personal data. A database table called vault, a browser PIN modal or a public ZIP is not sufficient implementation of these contracts.

Respect the locked-truth requirement

Use the owner-requested PIN as a secondary transaction gate under independently authenticated administrator MFA. The vault section defines the requested value and verification protocol. Lock original truth versions against in-place changes. Authorized corrections create a reviewed successor version, record its parent and approval evidence, and explicitly activate it for future attempts. Running attempts preserve their pinned source, paper, rule and key versions.

Apply this gate to protected truth metadata, published package mappings, key and rule corrections, protected release-policy changes and activation pointers. Apply ordinary ownership controls to student responses, notes and derived reports; these are not source-editing operations. The same policy must govern UI, APIs, workers, imports, migrations and support tooling.

Deliver evidence at each stage

- Build functioning persistence, authorization and scoring before displaying computed results.
- Prove source-lock denial and key isolation, then prove the 500,000-active-session workload with the stated bursts and failures.
- Keep vendor support, account quotas, actual capacity, recovery limits and incomplete integrations explicit. A deployed preview is a development milestone.

ACCEPTANCE / PRECAUTIONS

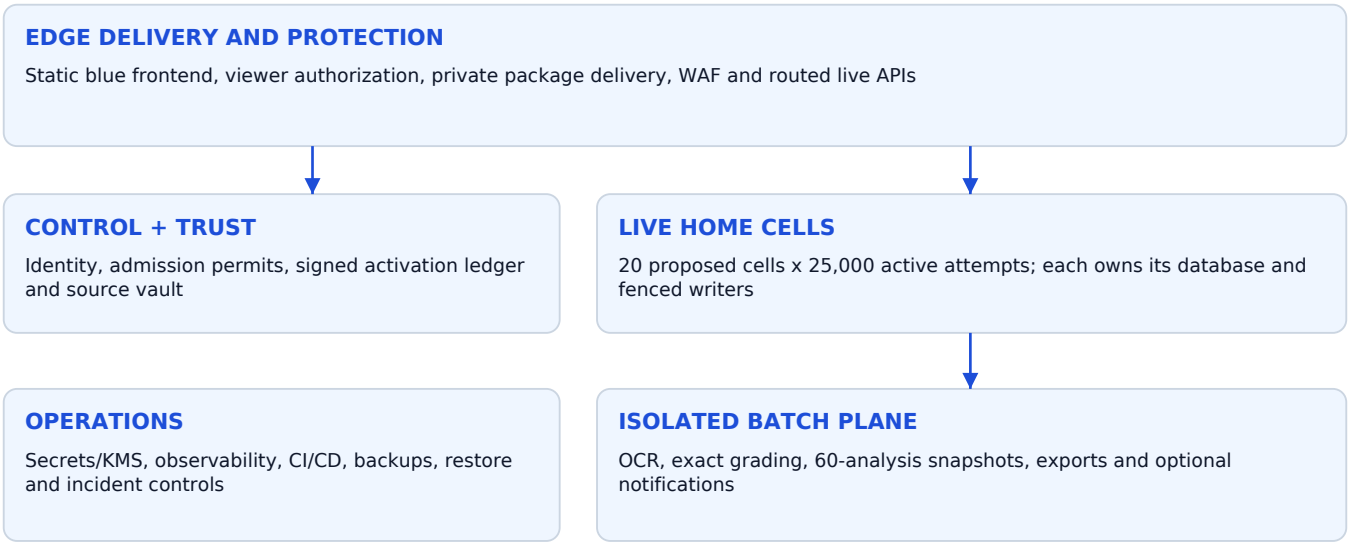
- Return the architecture, API and permission contracts before implementation begins.
- All final production capacity claims require the matching workload and fault-test artifacts.

System map: five separated operating planes

MASTER.02 | ARCHITECTURE MAP | RESPONSIBILITY BOUNDARIES

Implement five bounded planes. The global control plane distributes approved decisions; the live examination plane preserves already admitted students without depending on a global database transaction for every response.

REFERENCE TOPOLOGY - NUMBERS ARE DESIGN ASSUMPTIONS



Signed packages and permits are distributed before starts. Accepted answer writes remain inside the pinned home cell.

Control and trust planes

The trust plane holds reviewed sources and immutable versions. The control plane handles identity, entitlement, cell assignment and admission reservations. It distributes signed packages, verification keys and attempt permits before the exam window.

Live, batch and operational planes

The live plane gives every attempt a durable home cell and fenced writer. OCR, grading, analytics and exports use isolated batch queues. The operations plane manages secrets, releases, monitoring and recovery.

A failed report, search index or notification must not block an admitted student's commitment. Analytics reads derived snapshots instead of repeatedly joining live tables. Controlled degradation preserves existing exam obligations.

ACCEPTANCE / PRECAUTIONS

- The deployment diagram identifies every trust boundary and public endpoint.
- Block new admissions when reservation capacity is unavailable; preserve admitted attempts through control-plane outages.

Capacity contract: one shared numerical model

MASTER.03 | 500,000 ACTIVE CANDIDATES | DECIMAL UNITS

Use the following workload throughout architecture, tests and cost calculations. It is a proposed planning model. Record observed student behavior and revise the workload before claiming measured capacity. The nominal examination duration here is 10,800 seconds; actual exam durations remain configuration-driven.

Baseline request envelope

Operation / assumption	Calculated rate	Traffic lane
500,000 explicit commitments / 30 seconds	16,666.7 requests/s	Critical, durable
20% dirty drafts / 10 seconds	10,000 requests/s	Recoverable draft writes
500,000 independent leases / 60 seconds	8,333.3 requests/s	Critical control
Baseline live subtotal	35,000 requests/s	Before coalescing
500,000 telemetry batches / 30 seconds	16,666.7 requests/s	Separate bounded lane
Normal combined requests	51,666.7 requests/s	Excludes CDN asset hits
All dirty drafts / 5 seconds, plus commits + leases	125,000 live requests/s	Pathological scenario

Burst and sizing conventions

Reserve a tested two-times normal throughput envelope: 70,000 live requests/s and 33,333.3 telemetry batches/s. This headroom is not proof that a 125,000-live-request pathological workload succeeds; test that case separately, coalesce recoverable work and preserve eligible commitments. Requests, database transactions, response rows, event records and storage IOPS are different units.

Model 500,000 starts over 120 seconds as 4,166.7 starts/s and 500,000 submissions over 30 seconds as 16,666.7 submissions/s. A two-megabyte candidate package per student is one terabyte of edge delivery; over 120 seconds this is 8.333 GB/s or 66.667 Gbit/s before transport overhead. State decimal units, cache assumptions and whether assets are already prefetched.

ACCEPTANCE / PRECAUTIONS

- Measure per-cell write amplification and indexing overhead rather than equating API RPS with database TPS.
- Twenty cells at 25,000 active attempts is a proposed partition, not a measured capacity result.

Non-negotiable invariants and boundary rules

MASTER.04 | CORRECTNESS BEFORE CAPACITY

Build these invariants into code, storage permissions, schema constraints and tests. They define what must remain true when infrastructure is scaled, retried, restored or partially unavailable.

Truth and examination invariants

- A locked original, reviewed key, rule version or package manifest cannot be overwritten in place by any application route.
- One attempt binds one owner, home cell, immutable paper/configuration/key set, deadline policy and current fenced writer.
- A durably saved draft is separate from the committed response eligible for scoring. Background autosave does not invent Save or Review events.
- Only profile-eligible commitment events with trustworthy pre-deadline receipt may enter the frozen submission.
- Every acknowledgement states the actual durability level. A browser edit or volatile queue acknowledgement is not a durable-answer receipt.
- Question, subject and total marks reconcile using exact arithmetic; legacy results retain their grading lineage.

Operational invariants

One idempotency scope plus key maps to one validated request body and one durable outcome. A retry with a different body returns a conflict. A recovered writer accepts only the current fencing generation. A revoked grant or changed base version cannot authorize a new protected mutation. If audit, PIN validation or policy checks fail, protected source changes fail closed.

Signed candidate packages contain an allowlisted rendering projection only. Server key material and unreleased solutions are separate artifacts with separate identities and permissions. Authorization is checked before cached protected content is served. Admission uses reserved capacity, while post-admission overload controls preserve the existing examination obligations.

Zero-loss statements are limited to the failure modes covered by the tested durable replication policy. Asynchronous regional recovery may lose acknowledged facts within its measured recovery-point window; it must trigger reconciliation and an explicit fairness process rather than silently manufacturing missing answers.

ACCEPTANCE / PRECAUTIONS

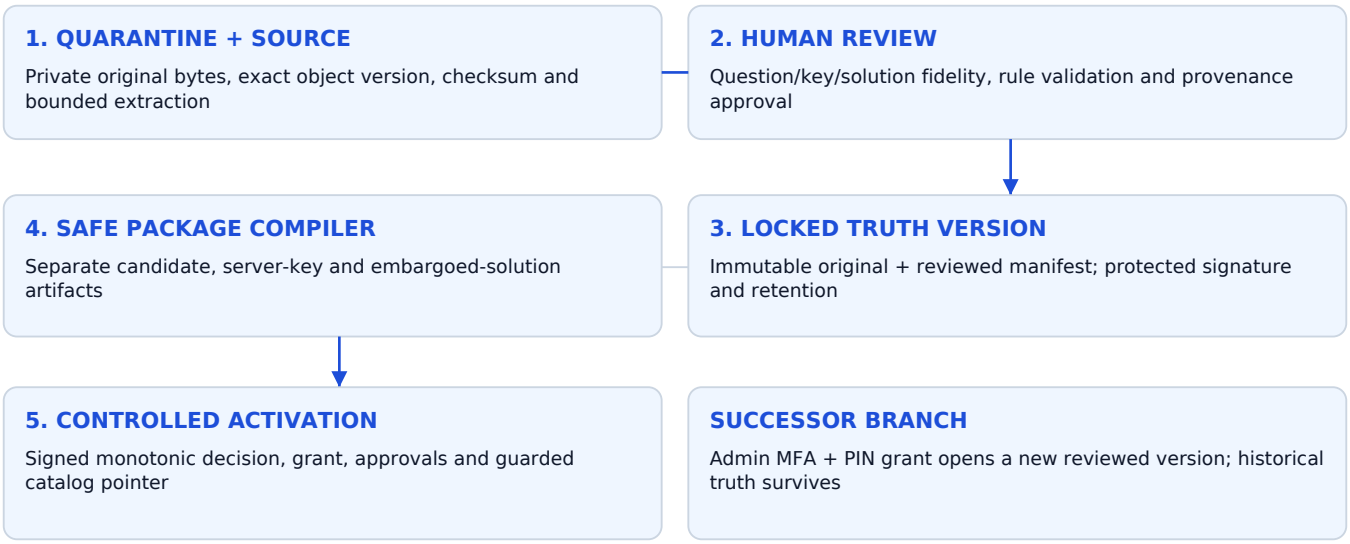
- [ ] Create a traceability row for each invariant: enforcing layer, failure scenario, expected result and proof artifact.
- [ ] Release is blocked by any source-bypass, wrong-score, cross-user access or covered-failure answer-loss defect.

Locked truth lifecycle: correction without erasure

MASTER.05 | SOURCE AUTHORITY | VERSIONED SUCCESSORS

Implement this source lifecycle for original PDFs, question interpretations, keys, solutions, rules and protected metadata. A successful PIN-authorized action creates accountable new work; the old locked truth remains a readable evidence record.

REFERENCE TOPOLOGY - NUMBERS ARE DESIGN ASSUMPTIONS



Current attempts preserve their pinned versions. A correction repeats review, locking, package checks and explicit activation.

From upload to publication

Quarantine each original with its object version and checksum. Review the extraction against exact pages and regions. A subject reviewer checks question, key, solution and rule fidelity. An authorized approver locks the reviewed truth; the compiler then creates separately controlled candidate, key and solution packages.

Successor workflow and safe consequences

A correction identifies the locked base, proposed fields, reason and evidence. Fresh admin MFA and a scoped PIN grant authorize a new draft branch. Approval, recompilation and activation follow protected operations. Changed diff or base version invalidates its approval.

The old source and grade remain evidence. A key correction creates a new approved key and an explicitly authorized rescore. Active attempts preserve their versions; support can trace every result to its source, rule, package and grader.

ACCEPTANCE / PRECAUTIONS

- The audit proves old and new checksums, request actor, approvers, grant scope, reason and activation time.
- An exam already running remains pinned even when a successor becomes active.

Feature modules and their authoritative stores

MASTER.06 | ALL FEATURES THROUGH VERSIONED CONTRACTS

Expose capabilities through typed APIs with server-owned permission checks. The browser displays and edits only its permitted projection. The following module map must appear in the repository and deployment documentation.

Core module map

Module	Authority / storage	Execution path
Identity, access, entitlement	Identity records and durable grants	Control API + cell permits
Source, OCR and editorial review	Quarantine + truth/version records	Private admin APIs + workers
Package and publication compiler	Signed immutable manifests	Compiler + protected activation
Exam profile and navigation	Pinned reviewed configuration	Cell reducer + client renderer
Responses, deadlines and submission	Home-cell transactional ledger	Critical cell APIs
Grading and corrections	Frozen attempts + approved keys	Isolated deterministic workers
60 analyses, cohorts and reports	Versioned derived snapshots	Batch aggregation + result APIs

Remaining application services

Implement question-wise review, tags, hints where a mode permits, notes, bookmark collections, revision tasks and the study plan through ownership-controlled resources. Provide source-linked support timelines, protected administration, object-aware search, reports, privacy export/deletion and optional commercial entitlements. Search indexes and caches are rebuildable projections; a search result must resolve to an authorized canonical record before protected content is retrieved.

Define how every feature behaves when a dependency is late or unavailable. Raw marks can be ready while cohort comparisons are pending; result release and solution release are independent permissions. Retried background jobs must publish the same logical result or a clearly versioned successor. AI suggestions enter editorial review and cannot modify locked truth, grant permissions or generate unverified exam rules.

ACCEPTANCE / PRECAUTIONS

- Map each visible screen and button to its route, service, owning store and tested denial case.
- Leave unavailable derived analyses explicit; do not use fabricated figures to make feature cards appear complete.

Shared names, mutation tiers and deadline closure

MASTER.07 | CROSS-SECTION IMPLEMENTATION CONTRACT

Use consistent identifiers and action semantics across all sections. Compile one API schema and one policy registry; do not create separate interpretations of immutable source, commitment or release inside different services.

Canonical identifiers and mutation tiers

Use source_id and source_version_id for original document evidence; truth_version_id for the reviewed locked interpretation; paper_version_id and exam_config_version_id for the attempt manifest; package_id plus package_hash for delivery; key_version_id and grading_version_id for scoring lineage; attempt_id, home_cell_id and writer_generation for live state. Item and Question are naming aliases, not competing entities. All timestamps persist in UTC.

Classify mutations as ordinary owner data, editable editorial draft, protected truth successor, protected activation/release, privileged account policy, or operational incident remedy. Ordinary student response APIs never ask for the administrator PIN. Protected truth actions require a payload-bound grant with actor, session, action, object, expected version, request hash, expiry and single-use nonce. Privileged identity changes require their own independent administrator controls and audit.

Trusted receipt and expiry closure

Distinguish browser action time, HTTP arrival, trusted cell admission time and transaction completion. Use the reference protocol in the scale section: one fenced, attempt-scoped transaction authority serializes authenticated command registration with deadline closure. Read fresh authority time after admission serialization; for PostgreSQL use clock_timestamp() after the lock, since now()/CURRENT_TIMESTAMP retain transaction-start time; register the full payload, request hash, idempotency identity and next contiguous accepted sequence transactionally. A mere pre-cutoff HTTP arrival is not acceptance. The close watermark contains only durably committed accepted commands, and FINAL follows processing through that watermark.

Acknowledge accepted commitments under the configured synchronous local durability policy; never silently degrade to asynchronous acknowledgement. Test a transaction admitted before cutoff that finishes afterwards and races expiry. A late identical idempotent retry can recover its durable outcome without changing the frozen answer. Alternative timing or durability policies require a reviewed configuration and new proofs; they cannot vary silently across cells. An overload rejection before durable registration remains a rejection; an unacknowledged local response is not silently credited.

ACCEPTANCE / PRECAUTIONS

- Demonstrate a pre-cutoff command whose processing finishes after cutoff under the selected policy.
- Race expiry, submit, lease takeover and retries; require one closed snapshot and one logical result.
- A transaction begun before cutoff but acquiring the attempt lock after cutoff is rejected.

Emergent responsibilities and production provisioning

MASTER.08 | CAPABILITY MATRIX | OWNER-OPERATED INFRASTRUCTURE

Use Emergent to create and iterate the application, contracts and deployment artifacts. Determine the actual hosting, database, networking and observability capabilities before selecting the production topology. Vendor documentation about automatic scaling does not prove this workload. [MASTER-S01]

Deliver a deployment capability matrix

For each service, identify the owner, selected vendor, supported region, provisioning method, account quota, credential model, cost unit, recovery boundary and test evidence. Rows must include frontend/CDN, WAF, identity, live load balancers, cell compute, home-cell databases, connection pools, caches, queues, source/object stores, KMS/secrets, batch workers, analytical storage, telemetry and backup destinations.

If the Emergent-managed environment supplies only a subset, deliver the missing production infrastructure through reviewed infrastructure-as-code and explicit integration endpoints. Use one contract-compatible application codebase; development can be small while production deploys isolated cells. Avoid claiming support for PostgreSQL, a specific queue, private networking or a particular managed identity unless the selected environment actually supplies it. Document and test any justified substitution.

Keep build inputs and deployments reproducible

Deliver environment templates containing secret names rather than values, container builds pinned by digest, migration scripts, policy documents, signing-key verification configuration and fixture generators. Promote the same tested application/package compiler artifact across stages. A production credential belongs in the selected secrets service; it must not appear in an Emergent chat prompt, frontend environment file or generated screenshot.

Specify which steps require an infrastructure engineer: account and quota preparation, network boundaries, durable replication policy, load-generator provisioning, security review, incident ownership and recovery rehearsal. Application generation and infrastructure provisioning are distinct tasks with distinct evidence. The final build report states which are implemented, provisioned, tested or still blocked.

ACCEPTANCE / PRECAUTIONS

- Provide an actual account-quota report and a dependency availability check before the 500,000-session run.
- Do not mark production readiness from a preview URL, node count or vendor marketing claim.

Implementation order and reviewable delivery contract

MASTER.09 | USE WITH THE SECTION-SPECIFIC BUILD PROMPTS

Read this master contract first, then implement the vault, packages/APIs, cell architecture and operating proof in that order. The operations section provides sequential execution prompts and the final acceptance dossier.

First executable vertical slice

Build one synthetic reviewed paper, one locked truth version, separate candidate/key/solution artifacts, one authenticated student attempt and one deterministic grade. Show source-page provenance and a successful successor branch through the PIN gate. Then exercise denial without a grant, rejection of a changed payload, old-version preservation, independent embargo release and an active attempt surviving publication of a new paper version.

Run the same API and reducer contracts in one cell, then provision the proposed partitioned layout. Preserve the stable home-cell routing and writer fencing while increasing traffic. A second cell must not require a second implementation of the exam rules. Validate each actual database replication and failover policy before its durability level is advertised in student acknowledgements.

Evidence packet expected from Emergent

- Repository and schema: module map, API specification, typed clients, migration order and exact locked dependencies.
- Trust and security: immutable object/version receipts, package scan, permission matrix, transaction-grant denials and signed audit evidence.
- Correctness: four supported exam fixtures, deadline races, draft/commit tests, exact scoring and controlled rescore lineage.
- Capacity: realistic active-session workload, load-generator capacity, quota report, cell benchmarks, multi-hour soak and submit/start burst results.
- Operations: dashboards, alert owners, regional recovery limits, restore report, release/rollback procedure and incident fairness controls.

Final operator decision

Release only within the tested admitted-user envelope. The final target remains 500,000 simultaneously active students; an intermediate pass is not a final pass. Keep answer-saving capacity reserved, stop new starts before exhausting it, and preserve accepted attempt facts during degradation. The release record must list achieved throughput, applicable failure coverage, remaining risks and named technical ownership.

ACCEPTANCE / PRECAUTIONS

- [ ] A new engineer can reproduce the build and the acceptance results from the handoff alone.
- [ ] The readiness register never labels unprovisioned, untested or placeholder services as complete.

Atomic admission: a permit becomes one timed attempt

MASTER.10 | START LOGIC | RESERVATIONS AND CLOCKS

A waiting-room token is not an examination start. Bind admission to a qualified home cell and turn a permitted reservation into exactly one durable attempt, with a reproducible start receipt.

Issue a bounded, verifiable permit

An admission permit identifies the owner, cohort or assignment, paper/configuration versions, home cell, allowed time window, activation generation, one reservation ID and signature. A client cannot choose a different home cell or increase its priority. A valid permit also requires the selected content and identity permissions. New admission uses a sufficiently fresh verified activation decision; stale or unavailable admission authority can restrict new starts without disrupting an already admitted attempt.

Consume and create in one transaction

In the home cell, verify the permit and current capacity, then atomically consume the reservation, create the uniquely scoped attempt and start receipt, and move reserved capacity into the active count. Use a durable idempotency record tied to the owner and request hash. A crash or lost response must leave either a recoverable created attempt or an unconsumed reservation; it must not create an uncounted attempt or charge two slots. The server returns the same attempt and deadline on retry.

Set started_at and deadline_at through the pinned scheduling policy, which explicitly distinguishes a common scheduled deadline from duration measured from an accepted individual start. Render these returned facts on the client. Entering a waiting room, downloading a package or receiving a permit must not start an individual attempt timer.

Release capacity without losing ownership

Reservation expiry releases only a reservation that was never consumed. It cannot free a slot belonging to an active attempt. Close or cancellation releases active capacity once in the same authoritative transition; repeated submit or expiry requests cannot decrement twice. A reconciliation job compares counters with reservation and attempt ledgers and raises an alert before unexplained drift is reused for admission. Browser disconnection or lease expiry alone does not silently create a new attempt or reset time.

ACCEPTANCE / PRECAUTIONS

- Race duplicate starts and simulate a crash at every reservation/attempt boundary.
- A consumed permit retry recovers the original attempt even after permit expiry.
- Start receipts, capacity counters and active attempt records reconcile exactly.

Master prompt: preserve the locked truth

VAULT.01 | VAULT / NON-NEGOTIABLE CONTRACT

Emergent, create an authoritative content vault that preserves the uploaded evidence and every approved examination interpretation. Treat the owner's PIN as an additional server authorization gate. Unlock means permission to propose a successor; it never means rewriting the protected original.

Define the protected boundary

Include original question PDFs, original solution PDFs, original byte hashes, question interpretations, extracted mathematics, answer keys, solutions, marking rules, question tags, strategy annotations, provenance, package manifests, release configuration, and activation history in the locked-truth boundary. Keep a manifest of the exact protected fields and resource types. Do not let a field escape protection merely because an administrator screen calls it metadata or an import script writes it indirectly.

Distinguish immutable evidence, editable draft branches, approved locked versions, and active-version pointers. A locked version remains available for historical verification after a successor becomes active. Existing attempts pin their own truth version, question versions, key version, tag version, scoring policy, and release-policy snapshot. A correction may create a separately explained rescore, but it must not silently replace the student's starting evidence.

Enforce protection outside the interface

Create one policy service that authorizes every protected mutation, including API calls, bulk imports, database jobs, storage writes, manifest generation, metadata edits, activation, rollback, release changes, and PIN-policy changes. Apply database privileges and storage IAM controls so a modal is never the only barrier. General web servers, OCR workers, analytics workers, and AI tools must have no direct protected-write credential. Deny an unrecognized operation by default. [V-06]

Explain the actual assurance

Use version-specific storage retention for original evidence and approved manifests, together with application-level immutability for semantic records. The retention duration is an operator-selected commitment, not an invented promise of eternal preservation. Display retention mode, retain-until date, object version IDs, and verification health in the administrator vault view. Treat cloud-account closure, lost keys, unavailable replicas, and revoked credentials as distinct availability risks that a content lock cannot eliminate.

ACCEPTANCE / PRECAUTIONS

- An unlocked proposal cannot mutate the bytes or semantic records of its parent.
- Every protected resource has a documented mutation route through the same policy service.
- Historical attempts resolve their exact pinned manifest without consulting the latest active version.

Original bytes, hashes, and question evidence

VAULT.02 | PROVENANCE / TRACEABILITY

Preserve the actual uploaded files before extraction, normalization, compression, or visual cleanup. Store derived assets separately. A rendered crop or reconstructed equation may improve delivery, but it cannot substitute for the evidence of what the uploader supplied.

Capture a defensible ingestion record

Create sourceEvidenceId, uploader identity reference, uploadedAt, declared rights basis, byte length, detected media type, storage bucket, immutable object key, objectVersionId, original SHA-256 digest, ingestion job version, scan status, and retention commitment. Compute the digest over original bytes server-side and verify it after transfer. Reject conflicting records for the same version rather than overwriting them. Record a new source identity when the owner uploads a revised PDF.

Separate integrity from authenticity. A matching SHA-256 digest establishes that the checked bytes match the recorded bytes; it does not prove who authored the paper, whether its claims are correct, or whether the uploader owns publication rights. Record provenance assertions with their evidence and reviewer status. Keep copyright and permission review distinct from the technical hash check.

Tie each interpretation to evidence

For every question and solution step, record the sourceEvidenceId, source page, coordinate system, crop bounding box, crop checksum, OCR region ID, extraction version, and reviewer decision. Link a key to its solution evidence and identify any disagreement between an answer list and a worked solution. Store mathematical corrections as proposed interpretations with a reason, never as edits to the original PDF. Require subject review before a conflicting key becomes authoritative.

Make verification reproducible

Create a canonical package manifest listing evidence versions, derived asset checksums, semantic record versions, policy IDs, reviewer approvals, and manifest schema version. Verify its checksums during publication and recovery. An incomplete evidence chain blocks a verified package claim. Provide an authorized reviewer with the original crop beside the interpretation, while keeping unreleased solution material out of student delivery. A source-file relocation must preserve its version identity and produce an audited locator update through the protected policy path.

ACCEPTANCE / PRECAUTIONS

- Re-download an original version and reproduce its recorded digest.
- A question, its OCR crop, its reviewed key, and its solution all resolve to evidence IDs.
- A hash match never produces an automatic authenticity or rights-ownership badge.

Owner PIN gate and scoped authorization grants

VAULT.03 | ADMIN AUTHORIZATION / BOOTSTRAP

Require a named privileged administrator authenticated through SSO and a strong enrolled MFA factor before the PIN challenge. The PIN is a secondary authorization requirement; it neither identifies a user nor replaces MFA, role checks, or review of a specific proposed action.

Honor the owner value securely

Initialize the owner's required PIN as 5524 through a server deployment secret named TRUTH_UNLOCK_PIN_BOOTSTRAP. Supply it through a secrets manager to the initialization service, never a frontend literal, repository default, public environment variable, PDF upload, API log, analytics event, or error message. Enroll a salted Argon2id verifier with a separately protected pepper and record verifierVersion; restrict bootstrap-secret access to that service. Retain this owner-selected value until an explicitly authorized rotation succeeds. The content vault must never contain the plaintext PIN. [V-03]

Use the hashing library's documented Argon2id implementation and benchmark current recommended work factors on the actual server. A short PIN has limited guessing entropy even when hashed; the pepper, privileged identity, throttling, and scoped grant remain essential. Protect secret transport with TLS and redact authentication request bodies before telemetry interception.

Issue an operation-bound grant

Verify the PIN only after the policy service confirms eligibility. On success, issue a server-stored, single-use grant bound to administrator ID, tenant, session, authentication assurance, resource ID, action enum, parent version, proposed-diff hash, reason hash, and nonce. Recheck permissions when consuming it. A proposed initial lifetime is 90 seconds; this is an operator-configurable product default, not an external security standard. An altered diff, resource, or action requires a fresh grant.

Throttle and fail closed

Use counters across administrator identity, tenant, and network signals, with safeguards against a shared-network denial of service. Proposed initial policy: three failed checks within ten minutes trigger a fifteen-minute challenge cooldown and an administrator alert. Do not reveal partial matches or whether a guessed user exists. Failure of the verifier, secrets manager, policy service, or grant store denies the protected action. MFA replacement and PIN recovery follow the controlled recovery workflow. [V-04]

ACCEPTANCE / PRECAUTIONS

- Correct PIN without SSO, MFA, or resource permission grants no access.
- Replay, expiration, changed diff, and changed resource each invalidate the grant.
- A logging test finds no plaintext PIN in requests, traces, failures, or browser bundles.

Least-privilege roles and contextual decisions

VAULT.04 | RBAC + ABAC / PERMISSION MATRIX

Define role permissions and contextual checks together. A broad Admin label must not grant every content, infrastructure, secret, and signing action. Make the role/action matrix reviewable and enforce the resulting decision on every request. [V-06]

Create the permission matrix

Implement the role/action matrix on the companion page as a reviewable policy artifact, with explicit allowed actions, denials, and resource context. A person's role identifies the possible actions; the tenant, relationship, version state, authentication assurance, and scoped grant determine whether a specific request is allowed. Reading assigned evidence does not grant publication or decryption authority.

Keep content approval, infrastructure administration, signing, audit management, and secret administration separate. A general operator cannot combine permissions from unrelated services into an unreviewed content update. Changes to the matrix and protected-resource allowlist themselves require the documented configuration review and authorization path. Export effective permissions, including workload grants, for comparison with this intended matrix.

Apply resource and relationship attributes

Check tenant ownership, assigned paper or subject, draft state, parent truth version, authentication freshness, grant binding, reviewer independence, classification, and release time. Recheck these attributes at command execution; a permission granted when the dialog opened may have been revoked before approval. A cross-tenant guessed object ID must fail even when the caller has a valid role in another tenant. Do not use possession of a signed URL as authority for new mutations.

Separate identities and environments

Use distinct workload identities for quarantine, extraction, draft storage, publication, scoring, delivery, signing, and audit export. Scope database privileges, IAM actions, and KMS grants to their actual task. Keep development test roles away from production secrets and locked storage. Make role grants, protected-resource classifications, and policy-service allowlists themselves controlled configuration changes. Compare effective privileges with approved intent, including inherited and wildcard access. Use an operator-approved tier enum; reject arbitrary overrides. A tier cannot disable the PIN gate, reviewer independence, or immutability.

ACCEPTANCE / PRECAUTIONS

- A content author cannot activate or self-approve a proposal.
- An AI worker cannot obtain a vault-write or signing credential.
- Cross-tenant object-ID substitution and stale-role requests fail server-side.

Permission matrix: roles, denials and contextual gates

VAULT.04A | ENFORCEMENT REFERENCE

Implement the following role matrix as a policy artifact. Each row is a maximum capability envelope, and the object, assignment, version and transaction-grant checks narrow it for each operation.

Role and action boundaries

Role	Allowed	Denied	Context
Student	Assigned questions; own report	Vault write; embargoed solutions	Own authorized attempt
Author	Draft branch edits	Locked write; self-approval	Assigned package; PIN grant
Reviewer	Inspect; record review	Direct write; activation	Assigned subject; PIN grant
Approver	Approve material diff	Edit diff; self-approval	MFA; PIN; exact hash
Policy service	Successor; activation event	Parent rewrite; bypass	Approved grants; guards
OCR / AI	Quarantine; candidate output	Vault; signing; secrets	Scoped extraction job
Operator	Cloud maintenance	Unreviewed content changes	Approved operational scope

Apply the decision at command execution

A valid role does not authorize a different tenant, an unassigned paper, a stale parent version or a changed diff. The grant is action-specific and single use. A denied action remains denied when requested through a worker, bulk import or direct API. Publisher or Activator is the explicit activation capability: it never follows merely from author or reviewer membership.

Independent review and effective policy

Content authors cannot approve their own material changes. Reviewers and approvers obtain grants for their permitted actions; grant issuance is capability-driven and must not require them to acquire publication authority. Export effective workload and human permissions, including inherited grants, and compare them with the approved matrix.

ACCEPTANCE / PRECAUTIONS

- Try each permitted action against an unauthorized tenant and require denial.
- An actor with review capability can obtain a review grant but cannot activate a package.
- Changing protected permissions follows the approved policy-change path.

Successor branches, approvals, and activation

VAULT.05 | IMMUTABLE VERSION LIFECYCLE

Implement unlock as creation of a new draft branch from an identified locked version. The parent remains immutable throughout editing, approval, publication, activation, rollback, and future recovery. Apply the same discipline to semantic fields and release behavior as to original files.

Define allowed protected operations

Use an explicit action enum such as CREATE_SUCCESSOR_DRAFT, UPDATE_SUCCESSOR_DRAFT, SUBMIT_DIFF_FOR_REVIEW, RECORD_APPROVAL, LOCK_SUCCESSOR, ACTIVATE_VERSION, ACTIVATE_PRIOR_VERSION, CHANGE_RELEASE_POLICY, and ROTATE_PIN_POLICY. Define the required grant and review class for each. Unknown actions fail closed. A draft edit references a stable parent, expected draft revision, reason, and operation-bound PIN grant; it never runs UPDATE against a locked semantic record.

Classify marking, answer keys, solutions, interpretations, tags affecting analytics, strategy rules, provenance, package manifests, activation, and result or solution release as material. Material changes require two distinct independent approvers who did not author the proposal. Each records the reviewed canonical diff hash, fresh authentication assurance, scoped PIN grant, and decision. Cosmetic changes still require the protected mutation gate; the policy service decides review class, not an arbitrary frontend flag.

Bind approval to exact content

Invalidate approvals when any reviewed field or referenced asset changes. Before locking, validate evidence hashes, complete question mappings, policy constraints, signatures, approver independence, and storage retention acknowledgements. Produce a successor manifest and new locked truth version. Store the relationship to the parent and the change rationale. Signing a draft alone does not activate it or declare it scientifically correct.

Activate explicitly and atomically

Require a separate activation decision against the approved manifest and current protected ledger head. Sign activation and release records binding exact manifest and policy hashes, release classes and effective times, monotonic activation_generation, parent generation, and reviewed approvals. Use fenced compare-and-swap against the expected ledger-head generation; commit that protected head before publishing the catalog pointer or admitting new attempts. The mutable database pointer is only a projection, never proof of authority.

Before new admission and control-plane restore, verify the signed decision and generation against the independently protected activation ledger. Reject replay to an older valid signed package or early-release pointer. Rollback requires a reviewed ACTIVATE_PRIOR_VERSION decision creating a new generation. Existing attempts retain their pinned original admission permit; do not add an online global gate to every answer save. Separate result and solution release decisions.

ACCEPTANCE / PRECAUTIONS

- Changing one reviewed tag or asset invalidates the previous approvals.
- Two approvals by the same person, or by the author, do not satisfy material review.
- Unauthorized database pointer replay or early release fails; reviewed rollback creates a new signed generation.

Object Lock retention and storage realities

VAULT.06 | WORM / AWS S3

Use S3 Object Lock with versioning for retained originals and locked manifests, with the retention mode and duration chosen by an authorized operator. Test the commitment before enabling it on production evidence; do not describe the owner PIN as an override of provider retention.

Choose an honest retention commitment

In compliance mode, a retained object version cannot be overwritten or deleted through ordinary account credentials, including the root user; the period cannot be shortened while it applies. In governance mode, a specially permitted actor can bypass retention with the explicit bypass request. The owner's application PIN changes neither behavior. Account closure remains a provider-level destructive risk outside an ordinary application guarantee. [V-01]

Select retain-until dates from the approved retention policy and record them per object version. Do not invent an infinite duration. A legal hold is an independently removable mechanism, not a substitute for permanent protection. After provider retention expires, the application's locked-version policy still prohibits ordinary mutation, but the administrator view must distinguish that policy protection from an active WORM commitment.

Protect version-specific references

Object Lock protects the identified object version; it does not freeze database rows, prevent new object versions, or prevent a delete marker from hiding the current key view. Store and retrieve exact version IDs rather than assuming an unversioned GET returns the authoritative evidence. New successors receive their own explicit lock settings and verification. Deny ordinary DeleteObject, DeleteObjectVersion, bypass, retention reduction, bucket-policy alteration, and legal-hold removal to application workloads. [V-01] [V-02]

Prevent configuration gaps

Use a policy that requires protected uploads to carry approved retention and encryption attributes, and verify the effective object retention after writing. Quarantine an artifact if its bucket lacks the expected lock configuration or retention cannot be read back. Replication destinations need compatible lock configuration and a monitored backlog. Separate governance bypass credentials from normal operations, require exceptional review, and alert on their use. Never make a general administrator's cloud console access a routine path around the content policy service.

ACCEPTANCE / PRECAUTIONS

- A retained compliance version remains unchanged despite a correct application PIN.
- A delete marker cannot conceal a pinned original from the verifier.
- Upload into a missing-lock bucket fails publication instead of receiving a false Locked badge.

Encryption, signing, and key survival

VAULT.07 | KMS / SECRETS SEPARATION

Protect confidentiality and provenance with different controls. Encryption restricts reading; manifest signatures support verification; hashes detect byte changes. None of these controls independently guarantees content correctness, publication rights, or survival of the cloud account.

Separate cryptographic purposes

Use distinct KMS key policies and workload permissions for source encryption, released-package delivery, secrets protection, audit signing, and manifest signing. Keep per-package encrypted data keys and bind the package identity and version into the supported encryption context. Do not grant a delivery workload source-vault decryption merely because both assets originated in the same paper. Store secret-manager identifiers in configuration, not raw credentials or the PIN pepper.

Sign the canonical locked manifest digest with an approved asymmetric signing key. Record key ID, signing algorithm, canonicalization schema, digest, signature, signing time, and approval references. AWS KMS signing verifies use of the corresponding private key and message integrity; it does not identify the human reviewer by itself. [V-05] Link the signing event to authenticated review records. Treat nonrepudiation as an evidentiary goal requiring reliable identity and key custody, not an automatic copyright certificate.

Protect key availability

Object Lock does not prevent the underlying encryption key from becoming unusable. A disabled key, changed key policy, inaccessible external key store, scheduled deletion, or completed deletion can make retained source bytes unreadable. AWS documents that deleting a KMS key can irreversibly prevent decryption. [V-02] [V-07] Do not describe retained ciphertext as a recoverable archive unless key access has also been tested.

Design guarded recovery

Deny key disablement, deletion scheduling, policy weakening, and secret destruction to ordinary application identities. Monitor those privileged events and alert designated responders immediately. Maintain inventory from protected object versions to required key IDs, regions, aliases, and recovery owners. Test restore and decryption from the replica environment. Do not promise export or backup of nonexportable managed KMS key material; use provider-supported resilience arrangements and preserve verification public keys and historical signing metadata. Crypto-shredding a protected source while retention applies is a destructive incident requiring explicit escalation, never a PIN-unlock operation.

ACCEPTANCE / PRECAUTIONS

- A replica restore decrypts the pinned source using its documented key dependency.
- Scheduling key deletion or disabling a source key generates an immediate protected alert.
- A valid signature never produces an automatic authorship or ownership claim.

Private origins and controlled student delivery

VAULT.08 | CONTENT ACCESS / CACHE BOUNDARIES

Keep the original vault private and deliver only an approved student package. Avoid letting a CDN cache, signed URL, browser payload, or convenience download bypass question assignment, solution embargo, tenant ownership, or protected-version boundaries.

Build purpose-specific packages

Generate question-only delivery manifests from approved locked truth. Put answer keys and solutions into separate protected objects and separate release-controlled manifests. The student package must not include hidden solution text, key-bearing JSON, unreleased OCR outputs, or the original solution PDF. If an original question PDF contains its own answer appendix, do not serve it as the running question asset; render and review a question-only derivative linked to that original evidence.

Keep source storage private and disable public access. For SSE-KMS S3 delivery, use CloudFront origin access control (OAC), not legacy OAI, with signed origin requests and distribution-scoped bucket and KMS permissions. [V-12] Authorize the attempt and exact package version before issuing asset links; never sign caller-selected object keys. OAC authenticates origin access, not the viewer. Redact bearer tokens from telemetry.

Constrain caching deliberately

Enforce a distinct viewer-access check before every cache lookup, including a warm hit: validate grant signature, expiry, scope, tenant, asset, package version, and applicable release decision. An origin-only authorization check is insufficient. Partition caches by tenant, package, language, release class, and asset version where relevant; keep personalized reports private. A proposed initial signed-link lifetime is sixty seconds, operator-reviewed rather than an external standard. Signed access remains bearer access during validity unless stronger revocation is implemented.

Handle release and withdrawal safely

Check result and solution gates independently; a score does not authorize solution delivery. Withdrawal stops new grants, revokes credentials where supported, and invalidates caches. Define and test a bounded new-request revocation policy, including issued-token validity and propagation delay. Already fetched browser assets, screenshots, and downloaded bytes cannot be recalled or erased by server revocation. Eligibility changes require the protected PIN, review, and activation workflow.

ACCEPTANCE / PRECAUTIONS

- A running attempt cannot obtain keys or solutions by guessing another package path.
- A question package extracted from a mixed PDF contains no answer appendix.
- Expired, omitted, and tampered viewer grants fail against an already warm cache; fetched browser bytes remain outside revocation.

Hostile PDFs, isolated extraction, and AI proposals

VAULT.09 | QUARANTINE / UNTRUSTED INPUT

Treat every uploaded PDF and every instruction embedded inside it as untrusted input. The file may be legitimate examination material while still containing malicious objects, expensive rendering instructions, remote references, misleading answer keys, or text aimed at an AI worker.

Isolate acquisition and parsing

Accept uploads into quarantine with an unpredictable server-assigned name, authenticated uploader, detected media type, size checks, and checksum. Do not trust the filename, declared MIME type, or PDF extension alone. Run antivirus and structural checks, then parse and render in a disposable sandbox with a read-only input mount, restricted output directory, bounded resources, patched libraries, and no internet access. Apply reviewed limits for page count, image dimensions, decompression expansion, CPU, memory, and job lifetime. [V-09]

Keep acquisition separate from parsing. A URL-fetch service, if needed, has an approved destination policy and blocks private addresses, metadata endpoints, redirect escapes, and DNS rebinding. Disable remote resource resolution in the parser itself. A content-disarm copy or safe rasterization is a derivative; preserve the quarantined original bytes under their own evidence identity. A rejected document can remain private evidence without becoming a published paper. [V-09] [V-10]

Constrain AI assistance

Send only job-scoped extracts to the AI stage. Treat embedded commands, comments, attachment text, and alleged system instructions as document content, never privileged instructions. The model may propose segmentation, transcription, tags, solutions, or suspected conflicts, but it cannot write the vault, activate a manifest, change a key, grant permissions, inspect secrets, or contact external destinations. Require structured output validation and an explicit human review decision.

Review semantic and rendering outcomes

Compare candidate question text, equations, diagrams, options, keys, and solution mappings with source crops. Flag ambiguous numbering, crop truncation, unreadable notation, key-versus-solution conflict, and duplicate questions. A high model confidence score is not approval. Preserve rejected and accepted extraction versions for reproducibility, with sensitive-data handling appropriate to quarantine. Promote only validated reviewed candidates into a draft branch, then use the standard protected successor workflow for publication.

ACCEPTANCE / PRECAUTIONS

- A PDF attempting metadata-service access or remote font fetch receives no network response.
- Embedded instructions to change the answer key remain inert source text.
- The AI worker's IAM identity cannot write any locked-vault object or activation record.

Append-only audit, chained roots, and validation

VAULT.10 | EVIDENCE / TAMPER DETECTION

Record protected decisions so an independent reviewer can reconstruct who proposed, authorized, approved, locked, activated, or attempted to alter a truth version. A database table named audit_log is not automatically immutable or independently trustworthy.

Define an audit event contract

For each protected operation, record eventId, monotonic sequence within its scope, authenticated actor ID, tenant, role and assurance references, action enum, target resource and parent version, canonical diff hash, grant ID reference, reason, approval IDs, decision, server time, correlation ID, and resulting manifest hash. Never record the PIN, pepper, session token, raw authorization header, signing credential, or full signed URL. Sanitize user-provided reasons against log injection. [V-08]

Append the operation and an audit outbox record in the same database transaction. A publisher transfers audit events to a separately administered append-only destination and verifies delivery. Protected mutations fail closed if their durable audit transaction cannot be recorded. Do not extend that fail-closed rule blindly to unrelated student answer saves; separate their availability and telemetry contract.

Anchor chains outside the writer

Chain events using canonical event digests and the previous accepted digest. Periodically sign a root containing sequence range, count, tenant scope, and last digest, then anchor it in independent retained storage under a different administrative role. A proposed initial anchor interval is five minutes; choose an operator-approved value and monitor lag. Hash chaining alone cannot detect a malicious rewrite of the entire chain if the attacker also controls the only root copy. Independently protect the activation/release ledger head and signing role from mutable catalog-database administrators. Compare signed generations and exact policy hashes during admission and restore; a valid old blob cannot authorize pointer rollback or early release. Protect this authority against ordinary database replay, while documenting limits under cloud-root or signer-code compromise.

Validate cloud and application evidence

Enable appropriate cloud control-plane and vault data-access logs and monitor missing delivery, permission changes, retention actions, key disablement, and account-level events. AWS CloudTrail integrity validation supplies signed digest files, but enabling it does not itself validate delivered logs; run a scheduled verifier and alert on gaps or failures. [V-11] Reconcile application events with cloud storage and KMS events. Restrict audit readers, record access to logs, and preserve pseudonymous actor references rather than unnecessary personal profiles.

ACCEPTANCE / PRECAUTIONS

- Editing, deleting, reordering, or truncating audit events causes verification failure.
- A forged audit chain or replayed activation pointer cannot pass independent-root and protected-head verification.
- Trace and logging tests confirm protected secrets are redacted before collection.

Retention, privacy, and controlled recovery

VAULT.11 | OPERATIONS / NO SECRET BYPASS

Keep the WORM source vault separate from student identities, attempt responses, support messages, and account documents. Retention commitments must preserve necessary evidence without turning ordinary privacy deletion into a conflict with immutable content storage.

Separate data classes and lifecycles

Store student profiles and personal administrative records in privacy-managed systems with approved deletion and retention workflows. Before promoting source files into retained storage, inspect whether they contain names, phone numbers, application identifiers, or unrelated personal data. Quarantine or redact into a separately reviewed derivative where appropriate, preserving the handling decision. Do not send a user's deletion request directly to a compliance-retained evidence object and claim the request succeeded when storage has rejected it.

Use actor references in manifests and audit records, with the identity mapping managed separately according to its policy. Account deletion revokes sessions and roles and removes eligible profile information; it does not erase approvals, alter a locked truth version, or silently reassign authorship. Record tombstoned identities in history so the evidence remains interpretable without retaining a full personal profile.

Recover through a documented control plane

Maintain independent backup copies of manifests, semantic snapshots, audit roots, retention inventory, and infrastructure configuration. Verify original hashes, signatures, version relationships, object retention, and key access during restore before accepting the rebuilt activation pointer. Rebuild lost database indexes from retained manifests without fabricating approvals. Maintain separately authenticated recovery operators and verify their independence from ordinary content roles.

Keep emergency access bounded

Break-glass procedures may restore identity-provider connectivity, recover service access, or stop publication, but they cannot remove the PIN requirement, shorten compliance retention, rewrite a locked parent, or silently activate an unapproved successor. Lost-PIN or MFA-factor recovery requires rigorously verified identities, independent approvals, a reason, protected audit, and an explicit new verifier-policy version. If those prerequisites cannot be met, protected mutation remains unavailable. Invalidate existing grants after rotation, preserve historical content versions, and notify designated administrators through the approved operational channel. [V-04]

ACCEPTANCE / PRECAUTIONS

- Deleting an administrator account leaves locked manifests and approval evidence intact.
- A restore proves hashes and key access before publishing its active-version pointer.
- A failed recovery verification leaves protected operations closed, with no hardcoded backdoor.

Vault threat cases and release evidence

VAULT.12 | ACCEPTANCE TESTS / HANDOFF

Deliver the vault as a tested control boundary, not a password dialog over mutable tables. All operational intervals and thresholds in this section are proposed defaults for operator review; the owner-selected PIN is the explicit exception.

Exercise authorization and replay attacks

Test correct PIN without authenticated administrator identity, missing MFA, stale role, cross-tenant object substitution, expired grant, reused grant, altered diff, changed reason, changed action, and concurrent grant consumption. Each unauthorized operation must leave locked truth unchanged and emit a sanitized denial event. Attempt direct database UPDATE, direct object overwrite, bulk-import mutation, and AI-tool activation. Verify storage and database credentials deny the bypass, not merely that the browser hides a button.

Test reviewer independence, approval invalidation after a tag change, explicit activation, compare-and-swap conflicts, rollback activation, and active attempts pinned to the old version. Prove that an owner-authorized unlock creates a draft successor while original bytes, hash, key, and manifest remain unchanged. Verify separate result and solution gates against guessed delivery paths and cache behavior. Add database replay to an older signed package and early-release policy replay fixtures; both must fail without a new reviewed signed generation.

Exercise storage and recovery failures

In isolated test accounts, test retained-version deletion, governance bypass separation, new-version creation, delete markers, missing lock configuration, retention readback failure, replica lag, disabled key, pending key deletion, secret-manager outage, and lost policy-service connectivity. Do not run destructive key or account tests on production evidence. Test whole-chain audit rewrites against an independent root and restores with a missing required key. Recovery must report its specific limitation instead of inventing an Eternal lock badge.

Return concrete deliverables

Provide the protected-resource inventory, role/action matrix, ABAC decision rules, IAM and database privileges, PIN enrollment and rotation runbooks, grant schema, change-review classification, successor lifecycle, canonical manifest schema, retention selection, key dependency map, audit verifier, recovery plan, and executable threat fixtures. Attach a primary-source register with verification dates: AWS Object Lock, KMS signing and deletion, CloudTrail validation, and OWASP authorization, password storage, MFA, logging, upload, and SSRF guidance. Source facts support the controls; they do not endorse this exact product design. Keep unresolved implementation assumptions visible until tested.

ACCEPTANCE / PRECAUTIONS

- The release report proves immutable parents, protected successor publication, and pinned attempts.
- A full restore validates evidence, signatures, audit roots, and decryption access.
- No frontend, worker, direct storage role, or emergency account can silently bypass the protected workflow.

Build authoritative packages and complete API contracts

API.01 | API-01 | COPY-PASTE EMERGENT PROMPT

Implement the content packaging and API boundary for the 500,000 concurrent-candidate design. Freeze reviewed truth, expose only permitted projections and make every important operation recoverable after a lost response.

Authoritative source and delivery separation

Create one internal authority chain: original source files, reviewed editorial candidates, approved item versions, locked paper manifest, signed package build and activated paper version. Keep the exact raw PDF hashes, provenance anchors, review decisions and internal signer identity. A PDF hash proves byte identity; it is not secret and does not prove correctness, copyright permission or authorship.

Produce four separate artifact classes: source-private, candidate-safe, server-key and embargoed-solution. Give them distinct storage prefixes or buckets, access policies and service permissions. Never publish a shared ZIP containing questions and keys with hidden=true. Browser encryption cannot protect an answer key if the candidate receives its decryption material.

API boundary and locked operations

Build canonical /v1/student, /v1/admin and /v1/integrations namespaces from a pinned OpenAPI 3.1 specification and validated JSON Schemas. Maintain route-specific ownership, role, workload tier, idempotency and response contracts. The database-derived owner and tenant decide access; a supplied user_id or guessed UUID never establishes ownership.

Treat PIN 5524 as a secondary gate for changing locked truth. Verify it only on the server, behind authenticated role authorization and throttling. Store its verifier as protected configuration; never hardcode it in browser code, page source, SDK examples or analytics. A successful PIN grants a short-lived permission for one exact action on one exact locked object/version, not general administrative access.

Completion requirements

- Generate candidate projections with an explicit field allowlist, and scan built assets for key material, annotated answers and private notes. Raw PDFs may contain answer headers and must not be distributed automatically.
- Keep attempts bound to immutable package, rule and order versions. A locked change creates a reviewed successor; active attempts do not adopt new truth silently.
- Implement all route groups on pages 8-11, durable command receipts, bounded jobs, typed SDKs and contract fixtures. Treat absent providers or placeholder endpoints as named incomplete dependencies.

ACCEPTANCE / PRECAUTIONS

- An ordinary student cannot fetch a key, raw source or unreleased explanation through any package or API.
- A valid PIN without the required role fails; a scoped grant cannot modify another paper.
- The final build includes real OpenAPI files, validated schemas and route-level evidence, not screenshots alone.

Preserve raw evidence and build four artifact classes

API.02 | API-02 | SOURCE BUNDLE AND STORAGE

Package content through reproducible build stages with explicit trust boundaries. Original source evidence and candidate delivery assets serve different purposes and require different permissions.

Private source bundle

Create a source bundle containing raw question PDFs, raw solution PDFs, separately supplied keys, sha256 hashes, uploader identity, rights record, extraction-run identifiers and reviewed source anchors. Record page index conventions, bounding boxes, rotation and transformations. Store malware inspection results and quarantine status beside the evidence; a private source bundle is never a student download.

Example source record: {"source_id":"src_7","role":"questions","sha256":"hex64","byte_size":832114,"rights_record_id":"rights_2","review_run_id":"rev_9","storage_class":"source-private"}. Keep object locators server-only. Inspect question PDFs for answer tables, marked correct options, handwriting and solution-bearing headers. Block candidate packaging until those regions are removed or approved safe derivatives exist.

Separate classes with least privilege

- candidate-safe contains reviewed stems, options, shared stimuli, accessible descriptions and question-only images in mode-specific projections. Strict live simulations exclude chapter/topic/difficulty/strategy tags from the bundle; permitted teaching or post-test review projections require their own release scope. Include no correct flags, private notes, explanations or raw PDFs.
- server-key contains exact key versions, response normalization policies, option-ID mappings, anomaly policies and grading references. Encrypt at rest with service-restricted decryption; only grading and authorized academic review services can read it.
- embargoed-solution contains reviewed explanations and released-review answer information. Keep it inaccessible before the exact server-enforced release condition, even if its assets are already prebuilt.
- source-private remains available only to scoped content operations and authorized investigations. Separate retention and legal-hold policies apply; accidental deletion must not remove evidence required to reproduce a correction.

Build output and attestations

Use deterministic canonical serialization and content-addressed asset hashes. Record builder version, schema hash, dependency versions, candidate projection hash, server-key package hash and solution package hash in an internal build attestation. Sign the attestation using a protected signing service with signer_id and key_id; verify before activation. Distinguish this internal signature from any claim that an exam authority signed the source PDF.

Rebuilding identical reviewed inputs should reproduce equivalent package contents. Any changed crop, locale variant, order or rule hash produces a new package identity. Retain build failures and prevent an incomplete package from becoming the current locked version.

ACCEPTANCE / PRECAUTIONS

- The candidate archive contains none of the server-key or solution files.
- A PDF answer header triggers a packaging blocker with its source region.
- A modified object checksum prevents activation even when its filename is unchanged.

Freeze manifests, ordering and public projections

API.03 | API-03 | MACHINE-READABLE PACKAGE IDENTITY

Publish immutable manifests that bind exact question content, rendering assets and exam behavior. Keep the full grading manifest internal and derive the student projection from a strict allowlist.

Internal manifest contract

Persist schema_version, schema_hash, paper_version_id, package_hash, rule_config_version_id, rule_hash, locale_variants, item_version_ids, stimulus_version_ids, question_hashes, order_policy_version, release_policy_version, projection_mode and build_attestation_id. Internal-only fields include key_version references, server-key package location, canonical-answer mappings and academic anomaly decisions.

Example internal item: {"slot_id":"slot_03","item_version_id":"iv_88","question_hash":"hex64","response_type":"single_choice","max_response_length":1,"key_version_id":"kv_9","canonical_option_ids":["opt_a","opt_b","opt_c","opt_d"]}. max_response_length and numeric constraints derive from the frozen response schema; do not interpret this example as a rule shared by every exam.

Candidate projection and stable mappings

Example candidate item: {"slot_id":"slot_03","display_number":3,"question_hash":"hex64","response_schema_id":"resp_single_v2","stem_blocks":[],"options":[{"display_option_id":"d_1","label":"A","blocks":[]}],"asset_refs":[],"locale":"en"}. The public projection includes renderable content and input constraints, never key_version_id, accepted values, correct flags or canonical-answer mappings.

At attempt creation, freeze slot order and displayed-option mapping server-side. Map submitted display_option_ids to canonical option IDs using that attempt's stored mapping. A locale switch changes reviewed text but retains slot and response identities. Refresh, resume, grading and correction all use the same mapping. Do not shuffle options again on navigation or derive grading from a displayed letter.

Integrity and presentation safeguards

- Hash the serialized question representation and its referenced asset hashes. Verify every package dependency exists and matches its checksum before locking.
- Exclude answer-bearing annotations, HTML data attributes, hidden DOM nodes, source maps, debug exports and renderer caches from candidate builds. CI inspection must examine the built package, not only source DTO definitions.
- Keep source labels distinct from display numbers and stable slot IDs. Shared passages reference frozen stimulus versions and appear with their dependent questions without duplicating grading state.
- Sign activation and release records with a monotonic generation, and verify them against an independently protected activation ledger. A general database or storage administrator cannot mint records or replay an older valid package by changing the current pointer. Separate signing privileges from ordinary deployment roles. Bind exact manifest and release-policy hashes, commit the ledger head before admission, and verify its generation on control-plane restore.

ACCEPTANCE / PRECAUTIONS

- [ ] Student package schema rejects a key_version_id field even if a developer adds it accidentally.
- [ ] Option shuffle, language change and resume preserve the original canonical grading mapping.
- [ ] Package activation fails if one image hash or rule hash disagrees with the reviewed manifest.

Authorize CDN requests without fragmenting cached content

API.04 | API-04 | EDGE DELIVERY AND REVOCATION

Validate access before returning a cached asset. Shared immutable content can cache efficiently while authorization remains candidate-specific and release-aware.

Authorization before cache lookup

Issue a short-lived signed cookie or token after checking candidate identity, attempt entitlement, package authorization and release scope. Validate signature, expiry, permitted package/cohort scope and revocation epoch at the edge on every request, including a cache hit. Protect the origin so users cannot bypass the edge through an object-storage URL. A package hash in a URL is not an access credential.

Let an authorization cohort identify candidates entitled to the same package and release scope. Tokens may remain subject-bound while authorizing that cohort. Do not place user_id, attempt_id or token bytes into the shared object cache key. Use package_hash + locale + asset_path + access_scope, separating question and released-solution namespaces.

Cache and expiry contract

Cache immutable question assets internally for long periods under content hashes. Set browser policy separately according to permitted local retention; use private or no-store for user-specific APIs, receipts, keys and unreleased explanations. Never apply a global cache-everything rule to /v1 paths. Avoid embedding short-lived presigned URLs inside long-lived cached manifests.

Bound token expiry by the authorized attempt or release window and the configured maximum TTL. Document allowable clock skew, edge-policy propagation and the measured revocation bound. A short token lifetime reduces exposure but does not provide instantaneous revocation. Version emergency deny lists or authorization epochs; establish fail-closed behavior for restricted packages when policy freshness exceeds its limit.

Scale and recovery behavior

- Warm representative package assets before large scheduled starts. Perform warm requests with permitted operational credentials without making origins public.
- Renew asset authorization through a cheap authenticated route using jitter and proactive renewal. A stale cookie should trigger a renewal flow; it must not clear saved responses or restart the deadline.
- Prevent a question-scope token from reading solution-scope objects. Configure cache variants so an authorized solution response cannot be reused for a question-only requester.
- Test edge authorization with cached objects, range requests, alternate hostnames and origin bypass attempts. Record hit ratio after validation rather than measuring an insecure public cache as success.

ACCEPTANCE / PRECAUTIONS

- A revoked or expired requester cannot read a warm cached object beyond the documented revocation bound.
- 500,000 candidates share immutable object cache entries without sharing account-specific responses.
- The origin refuses direct access even when the exact object key is known.

Define typed schemas, errors and global HTTP rules

API.05 | API-05 | OPENAPI AND JSON SCHEMA

Publish a pinned, executable interface specification. OpenAPI 3.1 describes HTTP interfaces and supports JSON Schema dialects; JSON Schema 2020-12 provides validation vocabulary [API-S01, API-S02].

Schemas and operation metadata

Create openapi.yaml pinned to a chosen 3.1 patch version and bundle all referenced schemas locally. Use a consistent dialect, explicit required fields, bounded strings and arrays, typed response unions and additionalProperties=false for security-sensitive request/projection objects. JSON Schema annotations alone do not enforce ownership, deadlines or academic correctness; implement those checks in application logic.

Every operation defines operationId, security, x-owner-policy, x-role-scope, x-workload-tier, x-idempotency-mode, body limits, success schemas and errors. Use integer sequence counters and decimal strings for exact authored numeric responses. IDs are opaque strings, dates are UTC timestamps and locale is an explicit field. Reject duplicate JSON keys and oversized nesting before business processing.

Common error and header contract

Example error: {"code":"lease_fenced","message":"Refresh the current attempt session.","request_id":"req_9","retryable":false,"details":{"required_epoch":8}}. Return 400 malformed input, 401 unauthenticated, 403 forbidden, 404 nonexistent or deliberately concealed object, 409 state/idempotency conflict, 412 stale If-Match, 413 oversize, 422 schema/domain invalid, 429 throttled and 503 temporary unavailable. Record internal causes privately; do not reveal another candidate's ID.

Every response carries X-Request-ID. Authenticated APIs use Cache-Control: private, no-store unless a documented immutable-public representation is intended. 201 supplies Location; 202 supplies Location and Retry-After for job polling; 429/503 include Retry-After when retry timing is known. Conditional editorial writes require If-Match and return a new ETag. Receipt-bearing mutations return X-Receipt-ID.

Precise status meanings

- A 202 means durably accepted work, not completed grading or completed export. A 200 batch response may contain item errors only under the explicitly defined partial-batch schema.
- Use an authenticated session with secure cookie settings and CSRF protection for browser mutations, or scoped bearer tokens for trusted integrations. Do not expose administrative credentials in a generated SDK.
- Capture resource authorization and allowed-field projection together. OWASP identifies both object-level and property-level authorization failures [API-S04, API-S05]; tests must inspect returned fields as well as route access.

ACCEPTANCE / PRECAUTIONS

- [ ] Generated request examples validate against the pinned schemas.
- [ ] Undocumented fields in a candidate response fail CI projection tests.
- [ ] 401, 403, 409 and 503 produce distinct recovery behavior in the client.

Make retries safe with durable command receipts

API.06 | API-06 | IDEMPOTENCY AND BATCH SEMANTICS

Define retry behavior per operation. HTTP method semantics do not make every POST safe to repeat; RFC 9110 distinguishes idempotent methods, and provider APIs demonstrate explicit key-based policies [API-S03, API-S06].

Key scope and payload identity

Require Idempotency-Key on start, commit, submit, package activation, locked edits, rescore, export and other non-repeatable commands. Scope the stored key by tenant, authenticated actor, operation and target object. Compute a canonical payload hash including relevant version/precondition and action fields. Reject the same scoped key with a changed payload as 409 idempotency_payload_conflict; do not execute the second request.

Atomically reserve the command and record its durable receipt with business state. A retry returns the original receipt and outcome; an in-progress command returns 202 and its receipt location. Store idempotency records in the authoritative datastore, not solely in a volatile cache. Define whether pre-execution validation failures reserve a key and keep that policy consistent in SDKs. A durable ingress 202 is pending work, not a Saved acknowledgement.

Lifetime and receipts

Use {"receipt_id":"rc_7","command_id":"cmd_4","command_state":"accepted","accepted_at":"...","payload_hash":"...","result_ref":null}. Retain start and submit receipts for the configured attempt/audit lifetime. A shorter deduplication cache TTL must not permit a second start or submission after eviction; business uniqueness constraints and retained receipts continue protecting the action.

Retries never extend the original deadline or recreate a writer lease. Poll the durable receipt after a lost response. Distinguish command_state received, accepted, applied and rejected from job_state queued, running, succeeded or failed. A transient worker failure does not reverse an applied command. Return original acceptance time; UI labels follow command state, not HTTP status alone.

Batch contract and client recovery

- Use all_or_nothing for answer commits: any invalid slot, stale epoch or sequence gap rejects the whole commit with its reason. Draft-only synchronization may use partial mode, returning a result for every operation_id and leaving failures pending locally. A commit 200 is allowed only after durable command application; pending ingress returns 202 and stays Saving until receipt polling proves application.
- Example draft response: {"batch_id":"b_2","mode":"partial","results":[{"operation_id":"op_1","status":"applied","server_seq":18},{"operation_id":"op_2","status":"rejected","code":"invalid_response"}]}. Never advance a global acknowledged watermark past an unresolved gap.
- Bound batch count and bytes. Retry only unacknowledged operations using stable operation IDs; use exponential backoff with jitter and preserve local pending changes until an authoritative receipt resolves them.

ACCEPTANCE / PRECAUTIONS

- A lost submit response followed by ten retries yields one submission and one receipt.
- A changed payload under the same key returns 409 without modifying state.
- Partial draft failures cannot erase the failed local answer or falsely acknowledge its sequence.

Attempt contract: commands, drafts and committed responses

API.07A | API.07 | API-07 | ATTEMPT COMMAND PROTOCOL

Keep durable transport state distinct from the exam's saved-answer semantics. The server controls state transitions, writer authority and deadlines according to the frozen rule profile.

Command and snapshot contracts

Each mutation carries attempt_id, operation_id, writer_epoch, expected_server_seq and payload. The server binds candidate identity from the session, rejects a fenced epoch and assigns monotonic server_seq after acceptance. Use per-attempt serialization or equivalent conditional transactions. Snapshot responses include state, deadline_at, server_now, committed responses, drafts, review flags, current epoch and last acknowledged sequence.

Example commit: {"operation_id":"op_24","writer_epoch":8,"expected_server_seq":31,"slot_id":"slot_3","response":{"kind":"choice","display_option_id":"d_2"},"response_hash":"hex64","action":"save_and_next"}. Validate the intended typed response and hash, atomically persist and commit it, update review/navigation state and return server_seq=32. Never copy a stale durable draft while newer local input awaits synchronization. An older draft upload cannot overwrite this committed response. Return 200 only with command_state=applied and a durably committed response; an ingress 202 remains Saving/Pending and requires receipt polling.

Client-facing acknowledgement states

A local selection remains a draft until a profile-eligible explicit action commits it. A durable command receipt and an applied committed-response acknowledgement are distinct states. Show Pending or Saving while an accepted command remains unapplied; show Saved only when the applicable committed response and its durability acknowledgement are available. The intended typed response must accompany an immediate Save even when background draft synchronization is still pending.

ACCEPTANCE / PRECAUTIONS

- An old writer epoch cannot replace answers after a new tab takes over.
- Synchronizing a draft does not make it a committed scored response.
- A transaction acquiring its attempt lock after cutoff is rejected; valid pre-cutoff admission can commit later.

Attempt contract: deadline closure and terminal receipts

API.07B | API.07 | API-07 | ATTEMPT COMMAND PROTOCOL

Apply the exact serialized-admission, fenced-writer and durable-closure protocol shared with MASTER.07 and SCALE.08. Receipt state and grading-job state are independent.

Leases, ordering and deadline decisions

Grant one active writer lease per attempt where required by the operating policy. A takeover increments the epoch and fences old tabs; renewing a lease never changes the attempt deadline. A stale tab can read a refreshed snapshot but cannot overwrite committed answers. Keep the mapping from display slot to canonical answer IDs server-side.

One fenced authority serializes registration and sealing under the attempt lock. After acquiring it, sample serialized_admission_at using PostgreSQL clock_timestamp(); now()/CURRENT_TIMESTAMP use transaction-start time [API-S08]. Eligible commands must register durably with fresh admission time before deadline. Commit may finish after cutoff while closing waits for the lock. A transaction begun before cutoff but locked afterward is rejected. HTTP/client timestamps are insufficient. Seal an accepted-command watermark; drain eligible commands before FINAL and record the committed sequence. Define one order for expiry, submit, BITS locking and incident adjustments.

Submission and grading separation

- Finalize submit after the accepted watermark drains. Return 202 with command_state=applied, immutable terminal submission receipt and grading job_state=queued. The SDK shows Submitted while grading remains pending; accepted ingress alone is not final submission. Enqueue grading through the transactional outbox.
- BITS bonus unlock/lock commands are separate atomic transitions governed by the frozen Volume 2 profile. A lock blocks prohibited earlier edits and remains in the snapshot after reconnect.
- Clear, review and save are distinct actions. Apply frozen profile semantics: where CLEAR empties only the draft, preserve committed answers until explicit SAVE. Keep raw input separate from normalized grading data and show errors.

ACCEPTANCE / PRECAUTIONS

- An old writer epoch cannot replace answers after a new tab takes over.
- Synchronizing a draft does not make it a committed scored response.
- A transaction acquiring its attempt lock after cutoff is rejected; valid pre-cutoff admission can commit later.

Expose catalog, identity and attempt operations (1/2)

API.08A | API.08 | API-08 | STUDENT ROUTE INVENTORY A

Implement the following student routes as real persistent operations. Route metadata is part of OpenAPI, and every nested attempt or package ID requires server-side ownership and entitlement validation.

Inventory legend and inherited contract

Inventory tuples are [role,tier,mode]. S means authenticated candidate acting on their own resources; L is the protected low-latency attempt lane, N is normal interactive work and Q is queued bulk work. R is read-only; K requires Idempotency-Key; V requires If-Match with a current ETag. Success codes are shown below; every route inherits page 5 errors, X-Request-ID and private no-store headers.

Use database ownership even when a route includes no user identifier. A candidate cannot select another actor through JSON fields. Catalog lists only permitted metadata; start enforces admission limits, paper activation, entitlement and scheduling. Return the frozen package/rule identity, deadline and writer epoch from start.

Bound catalog pagination and filter complexity. Admission and writer commands cannot invoke expensive report generation or synchronous PDF processing.

Required identity, discovery and start routes

- GET /v1/student/profile [S,N,R] 200: current profile and capabilities.
- PATCH /v1/student/profile [S,N,V] 200: approved profile fields only.
- GET /v1/student/catalog [S,N,R] 200: available papers and filters.
- GET /v1/student/papers/{paper} [S,N,R] 200: eligibility and instructions.
- GET /v1/student/exam-configs/{version} [S,N,R] 200: public frozen rules.
- POST /v1/student/attempts [S,L,K] 201: one admitted attempt.
- GET /v1/student/attempts [S,N,R] 200: owned paginated attempts.
- GET /v1/student/attempts/{attempt} [S,L,R] 200: authoritative recovery snapshot.
- POST /v1/student/attempts/{attempt}/asset-grants [S,L,K] 201: scoped authorization.
- POST /v1/student/attempts/{attempt}/lease [S,L,K] 200: acquire or renew.
- POST /v1/student/attempts/{attempt}/takeover [S,L,K] 200: increment fencing epoch.

ACCEPTANCE / PRECAUTIONS

- All eighteen operations enforce ownership and their documented state preconditions.
- Lease renewal and asset-grant retry preserve the original deadline.
- Clear, review and commit produce different recorded actions and receipts.

Expose catalog, identity and attempt operations (2/2)

API.08B | API.08 | API-08 | STUDENT ROUTE INVENTORY A

Implement this companion route family under the common role, lane, header, error, version and permission contract of API.05 and API.06, plus the specific protection rules in API.08.

Required answer and terminal routes

- POST /v1/student/attempts/{attempt}/drafts [S,L,K] 200: explicit batch results.
- POST /v1/student/attempts/{attempt}/commit [S,L,K] 200: atomic saved response.
- POST /v1/student/attempts/{attempt}/clear [S,L,K] 200: profile-specific clear behavior.
- POST /v1/student/attempts/{attempt}/review [S,L,K] 200: review flag transition.
- POST /v1/student/attempts/{attempt}/bonus-lock [S,L,K] 200: configured BITS transition.
- POST /v1/student/attempts/{attempt}/submit [S,L,K] 202: durable terminal receipt.
- GET /v1/student/receipts/{receipt} [S,L,R] 200: cheap command outcome.

Contract tests for this route family

For every operation, verify the authorized owner or assigned actor, the resource relationship, permitted fields, pinned version and required mutation tier. Duplicate mutation requests return the same scoped outcome; a changed request body under the same key is a conflict. Successful job acceptance remains separate from completed work. Each denial and asynchronous state must be represented in the typed SDK.

ACCEPTANCE / PRECAUTIONS

- All eighteen operations enforce ownership and their documented state preconditions.
- Lease renewal and asset-grant retry preserve the original deadline.
- Clear, review and commit produce different recorded actions and receipts.

Expose results, question insights and study workflows (1/2)

API.09A | API.09 | API-09 | STUDENT ROUTE INVENTORY B

Keep post-exam features off the attempt mutation lane. Results, analysis, solutions and personal study data require their own ownership and release rules.

Result contract and asynchronous readiness

Use the same S/N/Q and R/K/V legend from page 8. Return grading_state, grading_run_id, result_version, cohort_snapshot_id and generated_at on result representations. An accepted submission can have grading_state=pending; show that state honestly. Metric endpoints expose the complete metric catalog, including at least 60 supported analysis types, without claiming every metric is available for every attempt.

Each metric has method_version, input coverage, minimum sample rule, status and limitation text. Question-wise percentile or cohort comparison must identify eligible first-attempt populations and confidence limits. Solutions enforce the exact release scope on each access. A notes response must never grant access to unreleased explanations. Enforce independent result-release gates on results, every metric, question review, cohort comparison, study-plan inputs and report/export jobs, separately from solution-release gates. A prebuilt artifact remains inaccessible until its release condition passes.

Paginate drilldowns and attach the same result version to every page. A revised grade must not silently change an in-progress report.

Results and analysis routes

- GET /v1/student/attempts/{attempt}/grading [S,N,R] 200: pending or completed.
- GET /v1/student/attempts/{attempt}/result [S,N,R] 200: immutable result lineage.
- GET /v1/student/metrics [S,N,R] 200: sixty-metric catalog and methods.
- GET /v1/student/results/{result}/metrics [S,N,R] 200: availability-aware summaries.
- GET /v1/student/results/{result}/metrics/{metric} [S,N,R] 200: detailed calculation evidence.
- GET /v1/student/results/{result}/questions [S,N,R] 200: paginated item analysis.
- GET /v1/student/results/{result}/questions/{slot} [S,N,R] 200: response and timing.
- GET /v1/student/results/{result}/cohort [S,N,R] 200: qualified cohort comparison.
- GET /v1/student/attempts/{attempt}/solutions/{slot} [S,N,R] 200: released explanation.
- GET /v1/student/study-plans [S,N,R] 200: owned plan versions.
- POST /v1/student/study-plans [S,Q,K] 202: reproducible plan job.

ACCEPTANCE / PRECAUTIONS

- Metric unavailability includes a reason, rather than an invented number.
- Search and reports cannot expose another candidate or embargoed solution.
- Deletion reports actual completion and retained-record policy; 202 alone is not completion.

Expose results, question insights and study workflows (2/2)

API.09B | API.09 | API-09 | STUDENT ROUTE INVENTORY B

Implement this companion route family under the common role, lane, header, error, version and permission contract of API.05 and API.06, plus the specific protection rules in API.09.

Personal tools, reports and privacy routes

- POST /v1/student/notes [S,N,K] 201: create an owned note.
- GET /v1/student/notes [S,N,R] 200: owned filtered notes.
- PUT /v1/student/notes/{note} [S,N,V] 200: bounded sanitized note.
- DELETE /v1/student/notes/{note} [S,N,V] 204: owned note removal.
- PATCH /v1/student/study-plans/{plan}/tasks/{task} [S,N,V] 200: recorded task progress.
- GET /v1/student/search [S,N,R] 200: entitled released content only.
- POST /v1/student/practice-papers [S,Q,K] 202: reviewed-inventory custom paper.
- POST /v1/student/reports [S,Q,K] 202: report export job.
- GET /v1/student/jobs/{job} [S,N,R] 200: owned job and expiry.
- POST /v1/student/content-reports [S,N,K] 201: tracked academic issue.
- POST /v1/student/account-export [S,Q,K] 202: scoped personal-data archive.
- POST /v1/student/account-deletion [S,Q,K] 202: verified deletion workflow.
- GET /v1/student/jobs/{job}/artifact [S,N,R] 200: owned completed download.

Contract tests for this route family

For every operation, verify the authorized owner or assigned actor, the resource relationship, permitted fields, pinned version and required mutation tier. Duplicate mutation requests return the same scoped outcome; a changed request body under the same key is a conflict. Successful job acceptance remains separate from completed work. Each denial and asynchronous state must be represented in the typed SDK.

ACCEPTANCE / PRECAUTIONS

- Metric unavailability includes a reason, rather than an invented number.
- Search and reports cannot expose another candidate or embargoed solution.
- Deletion reports actual completion and retained-record policy; 202 alone is not completion.

Expose content intake, review and draft publishing (1/2)

API.10A | API.10 | API-10 | ADMIN ROUTE INVENTORY A

Build administrator APIs with tenant and subject scopes. Ordinary pre-lock drafts are editable by role; successor drafts derived from locked content inherit protection and require the exact scoped grant defined on page 11.

Administrative authorization and lane policy

A aliases the Vault Author; Rv aliases Reviewer. P is the Approver's explicit publication-request capability, not a broader publisher role. Policy service executes guarded successor, lock and activation commands. O is operations, G role administration and X scoped support. N/Q and R/K/V retain their meanings. Every capability includes tenant/object scope; no role inherits the others.

Uploading must yield actual-byte verification, asynchronous extraction and a review-required candidate. An OCR provider suggestion is not a key approval. Review endpoints reference exact candidate versions; edits invalidate relevant approvals. Build and validation endpoints return job receipts whose successful completion is required before locking.

Source downloads require a separate scoped capability. Ordinary editors can inspect safe previews without automatically receiving every raw key-bearing PDF.

Ingestion and academic review routes

- POST /v1/admin/uploads [A,N,K] 201: private upload manifest.
- POST /v1/admin/uploads/{upload}/complete [A,Q,K] 202: verified-byte ingestion.
- GET /v1/admin/jobs/{job} [A/Rv/P/O,N,R] 200: scoped stage progress.
- POST /v1/admin/jobs/{job}/retry [A/P/O,Q,K] 202: bounded new execution.
- POST /v1/admin/jobs/{job}/cancel [A/P/O,N,K] 202: acknowledged cancellation workflow.
- GET /v1/admin/candidates [A,N,R] 200: scoped review queue.
- PATCH /v1/admin/candidates/{candidate} [A,N,V] 200: corrected draft extraction.
- POST /v1/admin/candidates/{candidate}/review [Rv,N,K] 201: exact-version decision.
- POST /v1/admin/solution-links [Rv,N,K] 201: approved evidence-backed mapping.
- GET /v1/admin/tags [A,N,R] 200: versioned taxonomy.
- PATCH /v1/admin/items/{item}/tags [Rv,N,V] 200: reviewed assignments.

ACCEPTANCE / PRECAUTIONS

- Eighteen content operations return no cross-tenant sources or review counts.
- A stale draft ETag returns 412 with a safe refresh path.
- An extraction or build job can fail without slowing candidate response commits.

Expose content intake, review and draft publishing (2/2)

API.10B | API.10 | API-10 | ADMIN ROUTE INVENTORY A

Implement this companion route family under the common role, lane, header, error, version and permission contract of API.05 and API.06, plus the specific protection rules in API.10.

Paper, package and source operations

- POST /v1/admin/papers [A,N,K] 201: editable draft paper.
- PATCH /v1/admin/papers/{paper}/draft [A,N,V] 200: ordered draft manifest.
- POST /v1/admin/papers/{paper}/validate [P,Q,K] 202: complete readiness job.
- POST /v1/admin/papers/{paper}/build [P,Q,K] 202: four-class package build.
- GET /v1/admin/papers/{paper}/versions [A,N,R] 200: immutable lineage.
- GET /v1/admin/papers/{paper}/diff [A,N,R] 200: explicit compared versions.
- GET /v1/admin/sources/{source}/regions [Rv,N,R] 200: authorized source evidence.

Contract tests for this route family

For every operation, verify the authorized owner or assigned actor, the resource relationship, permitted fields, pinned version and required mutation tier. Duplicate mutation requests return the same scoped outcome; a changed request body under the same key is a conflict. Successful job acceptance remains separate from completed work. Each denial and asynchronous state must be represented in the typed SDK.

ACCEPTANCE / PRECAUTIONS

- Eighteen content operations return no cross-tenant sources or review counts.
- A stale draft ETag returns 412 with a safe refresh path.
- An extraction or build job can fail without slowing candidate response commits.

Gate locked changes, correction and fair incident actions (1/2)

API.11A | API.11 | API-11 | ADMIN ROUTE INVENTORY B

Protect the current locked truth with authenticated role checks, exact-version preconditions and secondary PIN grants. Operations tooling must preserve academic accountability and candidate fairness.

Scoped grant and successor activation

POST lock-grants verifies server-side PIN 5524 after authorized step-up authentication and role checks. Bind the single-use grant to tenant, actor/session, target/action, expected parent version, canonical diff/payload hash, reason hash and nonce. Require X-Locked-Change-Grant, Idempotency-Key and If-Match. Atomically consume it with the protected command receipt and audit record; changed payloads fail. Retry the same command through its receipt. Never log the PIN or verifier. Issue grants to any eligible Author, Reviewer or Approver for their allowed requested action; issuance grants no new role or capability.

Protected changes create a reviewed successor without mutating locked truth. Locking, activation and withdrawal require exact grants, as do locked tags, keys, configuration, release policies and capability/security policies. Validate then activate by compare-and-swap on the expected parent. Disabled roles or revoked sessions invalidate grant use. Verify signed manifest/policy hashes and activation generations against the protected ledger; reject replayed pointers. Document limits against cloud-root or application-code compromise. Store protected_parent_version_id on successor drafts; draft state cannot remove inherited grant checks from edits or draft-PATCH routes.

Lock, activation and correction routes

- POST /v1/admin/lock-grants [A/Rv/P,N,K] 201: exact scoped secondary grant.
- POST /v1/admin/papers/{paper}/lock [P,N,K] 200: freeze validated version.
- POST /v1/admin/papers/{paper}/successors [A,N,K] 201: protected successor draft.
- POST /v1/admin/papers/{paper}/activate [P,N,K] 200: protected atomic activation.
- POST /v1/admin/papers/{paper}/withdraw [P,N,K] 200: protected distribution withdrawal.
- POST /v1/admin/key-corrections [Rv,N,K] 201: protected evidence case.
- POST /v1/admin/key-corrections/{case}/approve [P,N,K] 200: exact correction decision.
- POST /v1/admin/key-corrections/{case}/rescore-preview [Rv,Q,K] 202: projected impact.
- POST /v1/admin/key-corrections/{case}/rescore [P,Q,K] 202: protected revised grading.

ACCEPTANCE / PRECAUTIONS

- A grant for paper A activation cannot authorize paper B editing or rescoring.
- Support cannot modify an answer or read keys through its diagnostic endpoint.
- Incident interventions preserve original deadlines, documented adjustments and affected-candidate evidence.
- Restoring a database pointer to an older valid package fails unless a reviewed ACTIVATE_PRIOR_VERSION decision creates a new signed generation.

Gate locked changes, correction and fair incident actions (2/2)

API.11B | API.11 | API-11 | ADMIN ROUTE INVENTORY B

Implement this companion route family under the common role, lane, header, error, version and permission contract of API.05 and API.06, plus the specific protection rules in API.11.

Rules, roles, support and incident routes

- POST /v1/admin/exam-configs [P,N,K] 201: reviewed successor rules.
- PATCH /v1/admin/exam-configs/{config}/draft [P,N,V] 200: unlocked rule draft.
- GET /v1/admin/roles [G,N,R] 200: scoped capability assignments.
- PATCH /v1/admin/roles/{assignment} [G,N,V] 200: audited role change.
- GET /v1/admin/support/attempts/{attempt} [X,N,R] 200: masked support snapshot.
- POST /v1/admin/incidents [O,N,K] 201: fair-treatment incident record.
- POST /v1/admin/incidents/{incident}/actions [O,N,K] 202: scoped recorded intervention.
- GET /v1/admin/audit-events [O,N,R] 200: permission-filtered immutable audit.
- POST /v1/admin/exports [P,Q,K] 202: authorized class-specific export.

Contract tests for this route family

For every operation, verify the authorized owner or assigned actor, the resource relationship, permitted fields, pinned version and required mutation tier. Duplicate mutation requests return the same scoped outcome; a changed request body under the same key is a conflict. Successful job acceptance remains separate from completed work. Each denial and asynchronous state must be represented in the typed SDK.

ACCEPTANCE / PRECAUTIONS

- A grant for paper A activation cannot authorize paper B editing or rescoring.
- Support cannot modify an answer or read keys through its diagnostic endpoint.
- Incident interventions preserve original deadlines, documented adjustments and affected-candidate evidence.
- Restoring a database pointer to an older valid package fails unless a reviewed ACTIVATE_PRIOR_VERSION decision creates a new signed generation.

Orchestrate jobs with an outbox and idempotent consumers

API.12 | API-12 | EVENTS AND ASYNCHRONOUS CONTRACTS

Use durable jobs for OCR, builds, grading, cohort statistics, sixty-metric analysis, study plans, reports and rescoring. Keep accepted attempt commands authoritative even when background processing is delayed.

Transactional event publication

Write the business mutation, receipt and outbox event in the same authoritative transaction. A dispatcher publishes pending events and records delivery progress. Consumers may receive duplicates; use durable event/inbox identities and unique business constraints. Do not promise globally exactly-once delivery. At-least-once transport with idempotent processing must produce one logical grading run or export per authorized job.

Example event: {"event_id":"evt_18","schema_version":"1.0","event_type":"attempt.submitted","tenant_id":"t_1","aggregate_id":"att_5","aggregate_seq":32,"occurred_at":"...","receipt_id":"rc_7","payload":{"snapshot_id":"snap_2","paper_version_id":"pv_9","rule_hash":"hex64"}}. Include references, not full keys, private PDF text or hundreds of answer assets.

State machines and ordering

Job states are queued, running, awaiting_review, succeeded, failed, cancelled and cancellation_requested. Include stage, checkpoint, attempt_count, heartbeat_at, next_retry_at, result_ref, error_code and owner_scope. A cancellation request is not cancellation completion. Retry only transient failures with bounded attempts; route exhausted work to a visible dead-letter queue.

Partition ordered work by attempt or paper aggregate, not a single global queue. Consumers reject stale aggregate sequence where relevant and fetch an authoritative snapshot when gaps appear. Use fencing tokens for worker leases so an expired worker cannot publish a newer result after its replacement finishes. Result publication uses conditional version checks. Pin executor/adapter version and input/output schema versions on every job and attempt. Rolling deployments retain compatible executors for admitted work; unsupported versions block new admission rather than reinterpret existing payloads.

Workload isolation and cheap outcomes

- Separate queues and compute budgets for ingestion, grading and optional analysis. Prioritize terminal grading without letting bulk OCR consume attempt admission or response-write capacity.
- GET receipt returns durable acceptance and current processing state without regenerating all metrics. Poll jobs using Retry-After, exponential backoff and jitter; avoid 500,000 clients polling every second.
- Checkpoint rescoring chunks and cohort recomputation. Store result versions and input lineage so rerunning an analytics task produces an explainable snapshot.
- Alert on oldest outbox age, accepted-to-graded lag, duplicate consumer rate and fenced worker writes. Queue depth alone does not reveal fairness or lost accepted submissions.

ACCEPTANCE / PRECAUTIONS

- Crashing after business commit but before event publish still leads to eventual grading.
- Redelivering the same event yields one logical outcome.
- An expired worker cannot overwrite the result produced by its replacement.

Verify optional integrations and payment events

API.13 | API-13 | WEBHOOKS AND EXTERNAL TRUST

Keep optional payments and external callbacks behind explicit integration contracts. Neither a browser redirect nor an unauthenticated callback establishes entitlement or completion.

Inbound webhook verification

When payments are enabled, verify the provider signature using the raw unmodified body, the configured endpoint secret and a bounded timestamp tolerance. Stripe's official webhook guidance describes signature verification and duplicate-event handling [API-S07]; other providers need their own verified contract. Select the provider deliberately and do not assume its fields or signing scheme match Stripe.

Persist provider account, event_id, event_type, API version, received_at, verification outcome and payload hash. Enforce uniqueness by provider/account/event identity. Accept valid duplicates without repeating entitlement changes. A signature proves provider-origin evidence, not that the referenced product or amount matches the intended order; reconcile those fields server-side.

Payment and integration route inventory

- POST /v1/student/orders [S,N,K] 201: server-priced intended purchase.
- GET /v1/student/orders/{order} [S,N,R] 200: authoritative settlement status.
- POST /v1/student/orders/{order}/checkout [S,N,K] 201: provider session reference.
- POST /v1/integrations/payments/{provider}/events [provider,Q,event-id] 202: verified durable receipt.
- GET /v1/admin/integrations [O,N,R] 200: masked configured integrations.
- POST /v1/admin/integrations [O,N,K] 201: allowlisted provider configuration.
- POST /v1/admin/integrations/{integration}/rotate-secret [O,N,K] 202: controlled secret rotation.
- POST /v1/admin/integrations/{integration}/reconcile [O,Q,K] 202: bounded reconciliation job.

Outgoing callbacks and lifecycle controls

Authorize an outgoing destination through an administrator allowlist and SSRF-safe fetch boundary. Sign event payloads with endpoint-specific secrets, include event_id, timestamp and schema_version, and retry with bounded backoff. Never let imported PDF instructions or student notes define a callback URL. Exclude keys, raw PDFs and personal responses from generic notifications.

Process asynchronous or out-of-order payment events against a settlement state machine and, where required, verify current provider object state. A return_url query saying success does not activate a subscription. Grants derive from settled server evidence and the order's frozen terms. If payments are disabled, document a free or manually authorized entitlement path rather than blocking unrelated exam features.

Retain an audit trail for refund, dispute and entitlement changes without storing payment credentials. Mask secret configuration responses. An integration secret rotation must support a documented transition window and replay tests, not a promise of instant worldwide replacement.

ACCEPTANCE / PRECAUTIONS

- A forged success redirect grants no new paper entitlement.
- Two valid deliveries of one provider event produce one settlement transition.
- An outgoing destination pointing at a private-network address is rejected.

Deliver SDKs, fixtures and release evidence

API.14 | API-14 | CONTRACT COMPLETION GATES

Generate usable clients and executable evidence from the same contract files. Completion means the APIs, packages and failure behavior work together under realistic load and adversarial requests.

Developer artifacts and compatibility

Deliver openapi.yaml, bundled JSON Schemas, candidate-package schema, internal manifest schema, event schemas and a route-to-feature coverage matrix. Generate typed TypeScript clients and server request validators from pinned tool versions. Do not generate a browser SDK that contains administrative signing keys, PIN constants or server-key DTOs. Provide separate student and administrative builds.

Include local fixtures with stable hashes, example requests, responses, typed errors and receipt replay scenarios. Mark examples synthetic. Version schemas compatibly: adding a required field or changing answer semantics needs a reviewed contract/version transition. Document deprecation windows and supported client versions. Runtime validation remains necessary even when TypeScript compiles.

Required contract and authorization fixtures

- Exercise every route against absent authentication, disabled role, wrong tenant, wrong owner, altered object ID, oversized request and unknown property. Inspect response fields for accidental key or personal-data disclosure.
- Build a source PDF with a hidden answer header, a tampered package asset, a wrong locale mapping and an answer-bearing source map. Assert that packaging or activation blocks each unsafe artifact.
- Replay start, commit, submit, activate and rescore after simulated lost responses. Change a payload under the same key, expire the cache while retaining the receipt and verify that no duplicate business outcome occurs.
- Race writer takeover, commit, deadline expiry, BITS lock and submission. Compare snapshots, receipts and immutable grading lineage against the frozen Volume 2 rule profile.
- Warm the CDN, then test unauthorized and revoked requests on cache hits and origin bypass routes. Measure shared cache hit ratio with authorization enabled.

Performance and final Emergent prompt

Load-test the actual payloads, batch sizes, asset distribution and polling rules used by 500,000 candidates. Report throughput, tail latency, error rates, database contention, queue lag and cache behavior by workload lane. Include retry storms and delayed background workers. Passing a small synthetic health-check load does not prove this concurrency target.

Implement the fourteen pages as one coherent package/API boundary. Return the coverage matrix, executable tests, package inventories, signer verification output, route inventory and pending dependencies. Show an attempt from catalog through durable submission and later analysis, then a PIN-gated reviewed successor activation that leaves that original attempt unchanged.

ACCEPTANCE / PRECAUTIONS

- Every required feature maps to a documented operation, schema, owner policy and tested outcome.
- Generated student SDK and candidate packages contain zero answer-key fields.
- Load evidence states tested concurrency and bottlenecks honestly rather than claiming untested capacity.

Half a million active examinations: the new capacity contract

SCALE.01 | SCALE.01 | REVISION MANDATE

Emergent, design this revision for 500,000 simultaneously active examination candidates. This replaces the earlier 10,000 launch and 50,000 stretch planning envelopes. It is a demanding engineering target, not a claim that a generated website, cloud account or prototype already provides this capacity.

Separate the product from its production infrastructure

Keep the polished React and TypeScript student application, real backend contracts and examination rules, but export a deployable production system. Do not route this workload through a single unmeasured preview server. Inspect Emergent hosting, repository access, environment configuration, background workers, network restrictions, database ownership and deployment automation. Mark every capability supplied, provisioned externally, substituted or unresolved.

Use a portable reference design: private object storage and CDN, protected HTTP ingress, stateless examination APIs, cell-local PostgreSQL with high availability, cell-local queues, separated grading and analytics workers, and an independent telemetry pipeline. AWS services are reference mappings, not required vendors or proof of suitability. An equivalent provider must demonstrate equivalent isolation, durability, quotas and observability.

Define the measured workload

The shared baseline is C = 500,000 active candidates over T = 10,800 seconds, representing a three-hour event. Distinguish active candidates from open browser tabs, registered accounts, requests, socket connections and completed attempts. Hold the response payload, question package size, event frequency and browser mix constant across capacity reports, or record the changed assumptions.

Produce a bill of materials with measured unit capacity and actual account quotas. Include database, backups, network, CDN, WAF, identity, storage encryption, queue and telemetry costs. Do not invent a fixed node count before representative benchmarks. Design a staged proof campaign that reaches the entire 500,000-candidate requirement and tests correlated failure at that scale.

- Protect accepted answer commitments ahead of exports, charts, imports and notifications.
- Keep draft persistence separate from scored answer commitment.
- Use server-authoritative deadlines and exact scoring values.
- Retain private source PDFs, answer keys and solutions outside public caches.
- Publish a dated capacity statement specifying the demonstrated workload and limitations.

ACCEPTANCE / PRECAUTIONS

- [ ] Require a working capability map from Emergent project to the production reference components.
- [ ] Keep all sizing, service objectives and recovery numbers proposed until measured.
- [ ] A preview URL and a vendor quota table do not constitute 500,000-user proof.

Twenty independent cells with bounded failure impact

SCALE.02 | SCALE.02 | DATA PLANE ISOLATION

Propose 20 production cells, each admitted for 25,000 active attempts. This 20 x 25,000 layout is an initial partitioning design to validate, not a demonstrated safe cell size. Reduce cell size or increase cell count if measurements require it.

Make each cell operationally complete

Give each cell its own examination API deployment, connection pool, durable database writer and high-availability standby arrangement, queues, grading workers, admission counters and operational dashboards. Separate database state and write capacity across cells. Twenty service deployments attached to one writer database do not provide twenty independent data planes.

Place a cell across availability zones rather than making one zone the entire live-attempt home. Use distinct infrastructure failure boundaries and, where justified, separate accounts or projects to reduce shared quotas and accidental permission impact. Document regional, account, cluster, network, key-management and DNS dependencies that still correlate failures. A cell design reduces some blast radii; it cannot eliminate a region-wide outage.

Keep global control out of live mutations

Use the global control plane for account provisioning, paper publication, source-vault administration, cell allocation, version manifests and fleet administration. Distribute approved immutable paper/profile versions and necessary authorization material to each cell before an event. An established attempt must save, commit, renew its lease and submit without a synchronous call to the global allocator, master content database or centralized analytics service.

A thin routing tier identifies the stable attempt home cell and forwards requests; it must not grade, perform deep policy evaluation or synchronously fetch global data on every mutation. AWS documents cell boundaries and a thin routing/control-plane separation [SCALE-S01]. The application-specific 20-cell proposal still needs isolation and saturation tests.

- Pin attempt_id, home_cell_id, paper_version, profile_version and routing epoch.
- Keep local service identities scoped to that cell and required shared assets.
- Roll out changes to one canary cell before advancing through the fleet.
- Disable an unhealthy cell for new allocation without rerouting its active writers.
- Expose cell-specific status and aggregate fleet status separately.

ACCEPTANCE / PRECAUTIONS

- [ ] Take the global allocator offline and demonstrate continuing saves for existing attempts.
- [ ] Saturate one cell and verify that unrelated cell write latency remains within its target.
- [ ] Map all shared dependencies honestly in the failure-domain inventory.

Stable home-cell routing, admission and safe rebalancing

SCALE.03 | SCALE.03 | PARTITION OWNERSHIP

Assign every attempt a durable home cell before its examination starts. Keep that assignment stable through refresh, retries and normal service failover. Do not use random request distribution across databases or user-controlled routing identifiers.

Publish a versioned allocation map

Choose an allocation key that distributes scheduled cohorts and large institutions evenly; do not place an entire popular paper or school into one cell merely because it shares a label. Use capacity-aware allocation for new attempts and a persisted attempt-to-cell record. Include routing epoch and a server-authenticated route token or equivalent lookup so students cannot claim another cell or attempt.

Routers need a replicated read-only allocation map and cached signing verification material. Define refresh, staleness and emergency withdrawal behavior. If the control plane is unavailable, existing stable routes continue within the approved policy; new allocations may pause. Reject stale or conflicting home assignments with a recoverable error rather than writing duplicate attempt state.

Move idle ownership, not active examination history

Rebalance future admissions by changing weighted cell allocation. Never migrate an active attempt between writers during an exam to improve a chart. For an idle account or completed attempt migration, stop new allocation for the selected scope, copy immutable versions and records, verify checksums and counts, fence old writes, publish the new routing epoch, and preserve a rollback boundary.

Do not treat the 20 cells as 20 units of spare capacity: at 500,000 admitted candidates they are fully allocated under the proposed envelope. If one cell fails, 25,000 assigned attempts need that cell's high-availability recovery or an explicit incident remedy. Remaining cells cannot silently absorb those active writers without state restoration, fencing and separately provisioned headroom.

- Allocate by measured free capacity, current active attempts and scheduled reservations.
- Prevent oversubscription using atomic per-cell admission counters.
- Reserve capacity for candidates already holding valid attempts.
- Keep waiting-room admission outside the running examination timer.
- Rate limit allocation-map changes and retain their audit history.

ACCEPTANCE / PRECAUTIONS

- [ ] Test stale router caches, two simultaneous starts and repeated creation idempotency keys.
- [ ] Demonstrate that a refresh reaches the same writer even after a routing rollout.
- [ ] Show a completed-attempt migration with old-writer fencing and no duplicated result.

The shared baseline: 35,000 live API requests per second

SCALE.04 | SCALE.04 | WORKLOAD ARITHMETIC

Use the following exact shared baseline in every infrastructure calculation. Frequencies are planning assumptions, not observations. Keep the uncoalesced upper envelope visible even if later optimization combines compatible requests.

Calculate each independent stream

Explicit eligible answer-commit events average one per candidate every 30 seconds: 500,000 / 30 = 16,666.7 requests per second. Dirty draft persistence assumes 20% of candidates have changes every ten seconds: 500,000 x 0.20 / 10 = 10,000 requests per second. Independent tab-lease renewal every 60 seconds adds 8,333.3 requests per second. Live examination API total: 35,000 requests per second.

Separate behavioral telemetry every 30 seconds adds another 16,666.7 requests per second. Therefore the combined baseline is 51,666.7 requests per second before retries, reads and other excluded streams. Reserve a 2x planning envelope: 70,000 live requests per second plus 33,333.3 telemetry requests per second, or 103,333.3 combined. Do not mix telemetry into the database answer-save budget.

Translate the model into cells and event volumes

Across 20 equally loaded cells, baseline live demand is 1,750 requests per second per cell: approximately 833.3 commits, 500 drafts and 416.7 renewals. Telemetry adds about 833.3 requests per second per cell equivalent, although it should use an isolated collection path. The 2x live envelope is 3,500 requests per second per cell.

Over T = 10,800 seconds, the baseline generates 180 million commitment requests, 108 million draft requests and 90 million lease requests: 378 million live requests. Telemetry generates 180 million requests, making 558 million combined. Request count is not automatically row count, disk IOPS or socket count; specify batching, transaction and retention behavior separately.

- Pathological case: 100% dirty candidates every five seconds creates 100,000 draft requests per second.
- Add ordinary commits and leases: 125,000 live requests per second in that case.
- Adding baseline telemetry gives 141,666.7 combined requests per second.
- Keep authentication, asset reads, starts, submits and retries as separate measured overlays.
- Do not apply 2x to a number that already contains the same headroom.

ACCEPTANCE / PRECAUTIONS

- [ ] Attach a calculation sheet with editable C, T, intervals, dirty fraction and retry factor.
- [ ] Make draft and commitment semantics remain distinct if transport coalesces them.
- [ ] Report cell skew and peak windows, not only the fleet-wide average.

Start, submit and question-delivery bursts

SCALE.05 | SCALE.05 | CORRELATED TRAFFIC

Treat scheduled examination traffic as correlated. Candidates can begin, refresh, request assets and submit together. An average three-hour request rate cannot size the first or last thirty seconds of an event.

Model the burst windows explicitly

Starting 500,000 candidates across 120 seconds creates 4,166.7 attempt-start requests per second. Submitting 500,000 across thirty seconds creates 16,666.7 submit requests per second; a shorter window is a harsher test. Overlay those rates on the live traffic still occurring in that period rather than replacing the baseline with the burst. Expiry sweeps and retrying uncertain submit receipts also need their own budget.

Keep submit acknowledgment short: validate and durably freeze eligible committed responses, write the submission receipt and grading outbox event, then return. Grade and build analyses asynchronously. Partition expiry scheduling by cell and deadline buckets so one global cron job does not lock or scan every attempt simultaneously.

Deliver packages from the edge

Assume a decimal question package of 2 MB = 2,000,000 bytes. Serving it once to 500,000 candidates transfers 1,000,000,000,000 bytes: 1 TB decimal. Across 120 seconds this is 8.333 GB per second, or 66.667 Gbps, before protocol overhead, retries and other assets. This is viewer-facing transfer, not a requirement to send every package from the application origin.

Use private, immutable, versioned packages behind authorized CDN delivery. Warm permitted caches and test regional cache misses, correct cache keys and entitlement verification. Do not pre-release visible examination questions or solutions while warming. The source vault is not the package-serving origin: approved derivative question assets may be delivered; original protected PDFs and key materials remain private.

- Record package-size distribution, asset count and compression behavior.
- Avoid one backend authorization round trip for every small image when a safe package grant suffices.
- Separate cacheable assets from uncached response mutations.
- Make cache misses bounded and observe origin request collapse.
- Include repeat download and mobile-network reconnection scenarios.

ACCEPTANCE / PRECAUTIONS

- [ ] Demonstrate the 1 TB transfer test with viewer throughput and actual origin load separately.
- [ ] Prove signed delivery cannot retrieve the private source or answer-key namespace.
- [ ] Measure start and submit bursts alongside ongoing commitments, not in isolation.

Compute sizing and autoscaling without invented node counts

SCALE.06 | SCALE.06 | MEASURED UNIT CAPACITY

Select compute after representative measurement. A node count unsupported by the actual API, database and encryption workload is guesswork. Keep capacity math explicit enough that an operator can reproduce the provisioning decision.

Measure a service unit, then size it

Benchmark one chosen node or task shape with production-equivalent TLS, authentication, request validation, connection pooling, durable database acknowledgment and telemetry overhead. Measure sustainable requests per second at the accepted latency/error targets. A no-op JSON endpoint is not an examination API benchmark. Define whether the measured unit is a host, pod, task or replica; never multiply pod capacity while overlooking shared host limits.

Use nodes >= ceil(required_RPS / (measured_RPS_per_node x utilization_target)) for the capacity calculation. Set required_RPS explicitly to the agreed baseline or headroom envelope. If measurements already represent operation at the chosen utilization target, use utilization_target = 1 in this formula to avoid discounting capacity twice. Calculate each critical service tier independently.

Add failure tolerance once

For a separate scenario losing 30% of equally capable hosts, require floor(0.70 x provisioned_nodes) x measured_RPS_per_node x utilization_target >= scenario_RPS. Solve that condition and take the maximum of the normal-capacity and failure-capacity requirements. State whether scenario_RPS is baseline or already 2x demand. Do not multiply another blanket failure margin into both formulas without describing the distinct risk.

Use scheduled prewarming before known examination starts, plus autoscaling driven by validated request rate, in-flight work, latency and queue age. Kubernetes supports custom metrics for horizontal autoscaling [SCALE-S02]; configure limits, stabilization and ready-instance behavior. Autoscaling is a feedback loop with provisioning delay, not protection against an instantaneous synchronized spike.

- Spread instances across availability zones and test host and zone loss separately.
- Bound backend connections before scaling replicas.
- Use dedicated resource pools for critical APIs, analytics and extraction.
- Preserve graceful shutdown and keep accepted mutations durable.
- Record warmup time, image-pull time and readiness before traffic admission.

ACCEPTANCE / PRECAUTIONS

- [ ] Publish benchmark script, code revision, unit shape, database shape and resulting capacity curve.
- [ ] Prove the 30% loss scenario without exceeding the admitted latency/error targets.
- [ ] Do not call CPU-only scaling sufficient when the database is the bottleneck.

Database write amplification, partitions and connection budgets

SCALE.07 | SCALE.07 | STORAGE HOT PATH

Model database work separately from API request rates. A small answer request can generate multiple row mutations, index writes and replicated WAL records. Adding frontend servers cannot increase an already saturated database writer.

Keep hot attempt data compact

Maintain current draft_response and committed_response separately, with independent revisions, explicit commitment-event type, fencing generation and authoritative receipt metadata. Do not overwrite the only accepted commitment history when a student edits an uncommitted draft. Retain sufficient immutable commitment facts to reproduce grading and disputes. Persist lease-generation issuance durably and define renewal persistence and failover behavior; do not quietly assume every renewal is free.

The baseline contains 180 million commitment requests across the three-hour event, or nine million per equally loaded cell. A design writing both an immutable commitment event and a current-response update already has two logical row mutations for each such request, before indexes and outbox work. Draft and lease mutations add more. Measure WAL bytes, IOPS, fsync latency, replication pressure and dead tuples rather than translating 35,000 requests per second into 35,000 IOPS.

Partition and pool deliberately

Shard writers by home cell first. Within each cell, partition large append-only event tables by suitable time buckets and, when justified by measurements, attempt hash. Keep partition count bounded and automate lifecycle creation before an event. PostgreSQL partitioning has key and index restrictions that must be checked for the selected schema [SCALE-S03]. Do not assume partitioning turns one writer into twenty independent writers.

Use bounded transaction connection pools with separate quotas for live mutations, grading and administration. Cap total pools across replicas; autoscaling must not open a new maximum-size pool per pod without a fleet budget. Keep transactions short and consistent in lock order. Put heavy cohort scans on separate analytical stores or replicas whose lag is acceptable.

- Store WBJEE partial-credit values as exact rationals or exact shared units.
- Preserve exact score totals and comparisons; round only for display.
- Reconcile table, index, WAL and backup storage growth separately.
- Measure autovacuum and checkpoint behavior during a multi-hour soak.
- Keep response ownership and idempotency constraints enforceable across partitions.

ACCEPTANCE / PRECAUTIONS

- [ ] Attach database throughput tests with the real draft/commit/lease write mix.
- [ ] Demonstrate that analytics saturation does not consume live connection reservations.
- [ ] Include partition rollover and storage exhaustion in failure testing.

Multi-AZ acknowledgments and authoritative deadline sealing

SCALE.08 | SCALE.08 | DURABILITY AND TIME

Implement a cell-local relational acceptance and seal protocol. HTTP arrival is not acceptance. Preserve complete authenticated commands durably before acknowledging them, and make the same fenced attempt authority serialize acceptance and closure.

Choose the durability contract explicitly

The selected reference acknowledges acceptance only after WAL flush on the primary and the configured synchronous, failover-safe standby or quorum. Verify that the chosen offering implements this durability contract; do not silently fall back to asynchronous acknowledgment when a standby fails. Quorum loss triggers bounded backpressure or a retryable fail-closed response. RDS synchronous standby and semisynchronous cluster offerings have distinct behavior [SCALE-S04, SCALE-S10].

Asynchronous cross-region replication has a separate nonzero loss window and regional RPO [SCALE-S05]. Keep regional disaster guarantees separate from the synchronous local acknowledgment contract. A timeout after transaction commit means outcome unknown to the client, not failure: recover the durable receipt and outcome on a later idempotent retry.

Serialize complete acceptance with the close boundary

For each authenticated command, execute an attempt-scoped transaction using its row lock or equivalent fenced compare-and-update. Recheck ownership, lease generation, profile and open state. Assign acceptance time with PostgreSQL clock_timestamp() after acquiring that lock; now()/CURRENT_TIMESTAMP retain transaction-start time [SCALE-S13]. A transaction started before cutoff but locked afterward rejects; a command admitted before cutoff may finish afterward while close waits. HTTP arrival and client time do not establish acceptance. Reject new commands at or after the deadline. Append the full validated payload, payload hash, idempotency key, acceptance time and next attempt-local sequence, and update last_accepted_seq in the same transaction.

Allocate no accepted sequence outside that transaction: rollback removes both append and counter advancement, preventing allocated-but-unwritten holes. The close operation uses the same attempt authority and transaction boundary; it stops acceptance and captures a watermark containing only committed accepted commands. Finalization drains and applies every contiguous accepted sequence through that watermark before entering FINAL. Delayed processing adds no answering time. Store final receipt and outcome durably; an identical late retry recovers them without accepting a new command.

- Partition authority by stable home cell and attempt; require no global write lock.
- Fence an old writer before promotion and prevent dual acceptance authorities.
- Separate draft persistence from eligible scored-commitment commands.
- Bound processing queues and preserve every already accepted full command.

ACCEPTANCE / PRECAUTIONS

- Crash between append and counter update: require complete rollback or complete durable acceptance.
- Race closure, quorum loss and late retries; FINAL must include exactly the sealed accepted sequence set.
- Start before cutoff, acquire the serialized lock after cutoff: require rejection.

CDN, load balancers, DNS and the quota ledger

SCALE.09 | SCALE.09 | NETWORK ENVELOPE

Provision network paths from measured requests, bytes and connection behavior. Inspect actual service quotas in the chosen account, region and pricing plan before launch. A published default quota is neither reserved capacity nor an achieved performance result.

Maintain a dated quota register

For the AWS reference, current CloudFront documentation lists 150 Gbps transfer and 250,000 requests per second per distribution, with stated exceptions for flat-rate plans [SCALE-S06]. Compare the selected plan and actual account approvals with the 66.667 Gbps package scenario, live requests, telemetry, asset request fan-out and retries. Keep safe margins and verify the provider's load-testing procedure.

Record ALB counts, target counts, listener/rule limits, regional capacity reservations, addresses, certificates, object-store requests, identity throughput, queue throughput, KMS calls and logging limits. ALB quotas are documented separately [SCALE-S07]. Request necessary changes well before a scheduled event and retain approval evidence. Do not create a new quota request during the opening minute of an exam.

Choose HTTP and connection routing deliberately

Use HTTP request routing where ownership, paths and protection policies require it. L4 transport distribution does not replace application authorization or home-cell selection [SCALE-S11, SCALE-S12]. Ordinary HTTP APIs with connection reuse are sufficient as a design option; the platform does not inherently require 500,000 simultaneous WebSocket connections. If WebSockets are added, measure memory, keepalive traffic, reconnection storms, termination behavior and provider quotas separately.

Plan private subnets, bounded east-west access, sufficient address space, DNS TTLs, resolver capacity and database endpoint changes during failover. Prefer private service endpoints for high-volume internal object or queue access where available. Monitor NAT port exhaustion, egress processing charges and connection churn. Never route public question-package traffic through backend NAT as an accidental bottleneck.

- Record edge throughput, origin throughput and private replication bandwidth separately.
- Verify signed CDN cache behavior without caching personalized API results.
- Test DNS failover with clients retaining old connections.
- Check cold-cache object requests and KMS demand; S3 scaling is not instantaneous [SCALE-S08].
- Measure candidate geography and mobile-network latency distribution.

ACCEPTANCE / PRECAUTIONS

- [ ] Provide a quota export and expiration/review date for each critical limit.
- [ ] Perform cold-cache and reconnect tests with realistic asset request counts.
- [ ] Prove the routing layer preserves stable home-cell ownership through endpoint failover.

Weighted admission, protected write capacity and hostile traffic

SCALE.10 | SCALE.10 | OVERLOAD CONTROL

Protect already admitted candidates when demand exceeds safe capacity. Apply weighted resource admission and differentiated priorities; a single request-rate cap ignores that a save, an export and a PDF extraction consume very different resources.

Reserve the critical path

Give eligible commitments, deadline sealing and submission receipts reserved compute, database connections and queue capacity. Draft saves and lease renewals remain essential but can use documented bounded coalescing that does not change scored commitments. Defer analytics refresh, bulk export, ingestion and optional notifications first. Provide clear service states rather than showing successful badges for discarded work.

Use per-cell tokens weighted by measured CPU time, transaction cost and memory footprint. Admission must consider database commit latency, pool wait, storage pressure and outstanding ingress receipts, not only healthy API processes. Stop new starts before the existing attempt path becomes unstable. Once a commitment is durably accepted, overload shedding cannot quietly delete it.

Separate abuse defense from honest capacity limits

Deploy DDoS protection, WAF policies, authentication abuse controls and server-side ownership checks. Shared school networks and carrier NAT can contain many legitimate candidates; a coarse per-IP limit must not block an entire classroom. Use authenticated account/attempt limits plus network signals and behavior analysis. Challenge suspicious sessions proportionately while keeping a documented support recovery path.

AWS WAF currently documents 100,000 requests per second per regional protection pack/web ACL; CloudFront WAF limits follow the CloudFront request envelope [SCALE-S09]. The 103,333.3 combined 2x model or 141,666.7 pathological combined model can exceed an unchanged regional arrangement. Check scope and actual approved limits; partition or request increases as appropriate rather than assuming one ACL covers every workload.

- Add jittered retry with retry-after guidance and bounded client queues.
- Reserve expiry/seal processing resources during submit storms.
- Keep telemetry collection on an independent bounded path.
- Distinguish dropped optional telemetry from lost examination commitments.
- Never loosen key protection or source-vault permissions to relieve performance pressure.

ACCEPTANCE / PRECAUTIONS

- [ ] Saturate the database while API health remains green and require admission to react.
- [ ] Test hostile traffic alongside legitimate shared-network candidate sessions.
- [ ] Verify rejected new starts do not consume an examination clock or payment entitlement.

Backup, restore and region-loss recovery of the cell fleet

SCALE.11 | SCALE.11 | RECOVERY AT REAL VOLUME

Rehearse recovery using production-equivalent synthetic data volumes. Restoring one tiny database does not establish recovery time for twenty busy cells, their object references and their admission/routing state.

Create a complete recovery inventory

Back up each cell database, immutable paper/profile manifests, approved derivative assets, commitment receipts, submission outbox state and grading versions. Keep source-vault backups under the vault's separate security controls; restoring an exam cell must not expose original PDFs or answer keys. Record cross-region copy lag, encryption keys, restore permissions, retention, delete obligations and provider restore concurrency limits.

Set proposed RPO and RTO separately for API restart, zone failover, cell database corruption and region loss. A regional target must include DNS/routing changes, writer fencing, secrets, object availability and validation before reopening writes. Measure actual achieved recovery points; backup job success and replica status are not proof that an attempt can resume safely.

Recover ownership before resuming mutations

Restore a cell to an isolated environment, reconcile accepted event sequences, latest draft revisions, committed revisions, lease generations and sealed submissions. Verify source checksums, object references and exact score reproduction. Re-dispatch committed outbox jobs idempotently. Fence the former writer and publish the new routing epoch only after the new writer is validated.

Do not copy a stale regional replica and silently allow it to accept current answers under the old lease. If regional recovery loses acknowledged commitments within the admitted RPO, identify affected attempts and execute a scoped fairness remedy. The remedy may be replacement testing or another approved approach; the system must retain its audit trail and avoid claiming those answers survived.

- Test one-cell restore and fleet-wide concurrent restore scheduling.
- Include restore bandwidth, object verification and database warmup time.
- Retain independent backup encryption and access monitoring.
- Practice a cold-cache restart with restored private asset grants.
- Run a full candidate start-save-commit-submit-review smoke test after restoration.

ACCEPTANCE / PRECAUTIONS

- [ ] Deliver a dated recovery report with volume, timeline, lost-window assessment and measured RPO/RTO.
- [ ] Confirm restored results match exact rational scoring fixtures.
- [ ] Keep all active-writer transitions fenced and auditable.

The 500,000-candidate proof campaign and release gate

SCALE.12 | SCALE.12 | EVIDENCE BEFORE CLAIMS

Prove the complete workload, including correctness under stress. The release gate is a reproducible evidence package, not a high number printed on a presentation or a vendor assurance disconnected from the actual application.

Build a distributed test campaign

Generate 500,000 unique synthetic candidates with stable home cells, valid sessions and realistic paper versions. Validate the load-generator fleet itself: connections, CPU, bandwidth, clock offset and request completion must not become the bottleneck. Include representative browser journeys alongside protocol-level throughput; use deterministic committed-answer fixtures so the expected final score is independently known.

Run staged tests at a single-cell level, a small multi-cell level and the full 20-cell fleet. Reproduce the 35,000 live plus 16,666.7 telemetry baseline, the 2x planning envelope, the 125,000-live pathological draft condition, the 120-second start burst, thirty-second submit burst and 1 TB package-delivery window. Run a three-hour soak with realistic retention and analytics drain.

Demand correctness and failure evidence

During representative phases, remove 30% of application hosts, fail a selected availability-zone path, delay a database, saturate noncritical workers, lose a submit acknowledgment, refresh clients, rotate routing configuration and force cold caches. Verify accepted commitment preservation, unchanged home-cell ownership, deterministic deadline sealing and exact score reconciliation. Track how many candidates fail each journey, not only HTTP success percentages.

Report p50/p95/p99 latency by route and cell, error categories, database commit time, pool wait, accepted event backlog, queue age, replication lag, key-denial tests, CDN viewer/origin traffic and cost. Proposed service targets must have explicit numerical pass/fail thresholds approved in the operations specification. Report failed scenarios and maximum demonstrated concurrency without relabeling them as a pass.

- Evidence: scripts, workload manifest, code hash, cloud topology and actual quota approvals.
- Evidence: raw summaries, dated graphs, integrity reconciliation and recovery reports.
- Evidence: capacity equations using measured unit performance and explicit utilization.
- Gate: no lost accepted commitments or unauthorized key/source access in required scenarios.
- Gate: public admission stays within the demonstrated workload and conservative safety envelope.

ACCEPTANCE / PRECAUTIONS

- [ ] Do not claim 500,000 simultaneous active exams until the end-to-end campaign passes.
- [ ] Keep a smaller live rollout while the full target remains unproven.
- [ ] Provide a concrete operator-ready capacity and limitations statement.

The operational contract for 500,000 active students

OPS.01 | OPERATIONS / MEASUREMENT AND RELEASE CONTRACT

Emergent, qualify 500,000 concurrently active student sessions for a full three-hour exam workload. This target means distinct authenticated attempts that read questions, change responses, commit answers, renew leases and submit; it does not mean 500,000 registered accounts or idle virtual users. Treat every number below as a proposed acceptance contract requiring measured evidence and operator signoff.

Define the qualified deployment

Use the proposed 20-cell layout with at most 25,000 admitted active attempts per qualified cell. Freeze the cell resource specification, database variant, connection pools, dependency versions, route distribution and exam mix in a run manifest. Qualify two times that cell's baseline request mix as proposed throughput headroom, then test the defined bursts and failures separately. Do not describe spare capacity, autoscaling limits or a successful small test as guaranteed 500,000-student capacity. Name the exact database deployment variant; Multi-AZ instances, Multi-AZ clusters and Aurora must not share an unverified replication promise.

Keep admitted sessions protected when reducing new admissions. Capacity approval must identify which failure modes preserve acknowledgements and which regional disasters can lose asynchronously replicated data. A page declaring an answer saved must correspond to the exam engine's durable committed response, not a draft, cache write or merely accepted queue message.

Set measurable proposed SLOs

For valid commit requests, measure in-region server ingress to durable acknowledgement: p95 <= 300 ms and p99 <= 1,000 ms, including queueing and database persistence. Separately measure client-observed round-trip time and rendered save feedback on representative Indian networks; report those values without relabeling server processing as total student latency. Operators must approve the separate client experience budget before launch.

Under the defined submission burst, require durable submission-receipt p99 <= 2 seconds. Measure receipt-to-correct-core-grade readiness at p95 <= 60 seconds and p99 <= 120 seconds; a visible pending grade is permitted while that job runs. Server errors must be <= 0.1 percent across the run and within each predeclared critical evaluation window. Integrity constraints have zero tolerance: no wrong grading, duplicate score effects or lost acknowledged commits within the covered failure envelope.

Publish the evidence boundary

Distinguish server correctness, API performance, browser behavior, connection capacity and disaster recovery. Publish actual achieved load and shortfalls. Never call an interrupted, under-generated or reduced-activity run a pass. Keep the same endpoint and timing definitions from generator scripts through dashboards and the final release decision.

ACCEPTANCE / PRECAUTIONS

- Version the workload, SLO definitions and failure envelope.
- Require a qualified-cell and global run decision.
- Preserve admitted-session write capacity during admission throttling.

Translate active students into real traffic

OPS.02 | OPERATIONS / EXACT WORKLOAD ARITHMETIC

Build a versioned workload calculator before selecting infrastructure. The required baseline is 500,000 active students, a commit operation every 30 seconds, dirty-draft traffic from 20 percent of students every 10 seconds, and a lease renewal every 60 seconds. Model telemetry as a separate pipeline and add content reads, starts, submissions and retries explicitly.

Carry the baseline without hiding traffic

Commit rate = 500,000/30 = 16,666.7 requests/second. Draft rate = 500,000*0.20/10 = 10,000 requests/second. Lease rate = 500,000/60 = 8,333.3 requests/second. The shared live API baseline is therefore 35,000 requests/second. Telemetry batches every 30 seconds add 16,666.7 requests/second on a separately limited collector; the combined dynamic rate is 51,666.7 before other reads or bursts.

Over 10,800 steady-state seconds, the baseline produces 180 million commit operations, 108 million draft operations and 90 million lease renewals: 378 million live API operations. Telemetry adds 180 million batches. Distinguish repeated or no-change commits from distinct graded answers; a periodic write operation must not manufacture additional responses or score contributions.

Specify synchronized phases

Admitting 500,000 sessions over 120 seconds requires 4,166.7 start requests/second, plus authentication and initial content work. Submitting them over 30 seconds requires 16,666.7 submissions/second. If submissions overlap all baseline work, the live API demand is 51,666.7 requests/second; with telemetry still active it reaches 68,333.3, before retries and asset traffic. Test that overlap rather than subtracting work optimistically.

For a 25,000-student cell, the normal live API mix is 1,750 requests/second, with 833.3 telemetry batches/second separately. Two-times baseline headroom means qualifying 3,500 live requests/second at that mix; it does not authorize 50,000 concurrent attempts in that cell. Test uneven routing and a hot-cell burst, because global averages can conceal local saturation.

Compute data and grading demand

Use bytes/second = sum(endpoint_rate * measured_request_plus_response_bytes), with compression and protocol overhead separately identified. Initial content demand is students * package_bytes, adjusted by measured edge hits and origin misses. Grading demand is students * items_per_attempt / target_completion_seconds; a 75-item example over 120 seconds requires 312,500 item evaluations/second globally. Recalculate for the actual adapter mix and partial-credit logic.

ACCEPTANCE / PRECAUTIONS

- Keep telemetry capacity separate from answer persistence.
- Record payload sizes and retries in the calculator.
- Check per-cell rates, global rates and total operations reconcile.

Prove sessions, browsers and connections separately

OPS.03 | OPERATIONS / REPRESENTATIVE CLIENT EVIDENCE

Implement three complementary test tracks. Stateful HTTP actors prove the 500,000-session application workload; a representative real-browser subset proves rendering and client behavior; a separately sized connection exercise proves network and gateway limits. A large count in one track cannot substitute for evidence in another.

Create authentic HTTP actor fixtures

Give every synthetic student an independent identity, attempt identifier, answer-state machine, lease, adapter version and deterministic seed. Exercise question reads, single-choice and numerical input, supported multiselect rules, clears, review flags, accepted commitments and the final deadline transition. Randomize realistic phases while retaining a separately defined synchronized burst. Compare actual per-actor activity and response changes with the expected manifest. Use distinct idempotency keys for new operations; reserve deliberate duplicates for duplicate scenarios. Never turn the baseline into cached retry receipts. Measure new, changed, cleared and no-change responses separately.

Use fixed expected-response fixtures, including correct, wrong, partial-credit, unanswered and invalid-item cases. Maintain an independent expected ledger that can be joined with server receipts after the run. Every accepted commitment must appear with its sequence and payload hash; every attempt/key revision must produce one scoring effect despite duplicate job deliveries. Merely refreshing sessions or returning HTTP 200 is insufficient.

Run browsers for client-specific failure modes

Start with a small validated browser set, then an operator-approved representative subset, for example 500 to 2,000 browser instances if generator capacity permits. Cover desktop, Android phone, tablet, slow connectivity, device backgrounding, image-heavy content, accessible mathematics and numerical keyboard interactions. Measure visible save states, reconnection, source-asset rendering and submit feedback.

Record the browser subset size and network profiles. It proves those tested experiences, not 500,000 full browser renders. Repeat critical interactions during HTTP saturation so the client and API evidence overlap. Include the committed-A/local-draft-B fixture: navigation, periodic draft persistence and final submission must not accidentally score draft B unless the configured action explicitly commits it.

Size connection proof from observed topology

Count browser connections per origin, keep-alive behavior, HTTP/2 multiplexing and any persistent channels. Load generators can multiplex many actors over a few sockets, so 500,000 logical sessions do not prove 500,000 concurrent TLS sockets. Exercise the measured connection inventory and churn separately, including handshake peaks, gateway file descriptors, NAT capacity and stale connection recovery. If a 500,000-socket claim is made, retain direct connection-count evidence.

ACCEPTANCE / PRECAUTIONS

- Verify distinct actors remain active throughout the soak.
- Join accepted receipts to independent expected responses.
- Publish HTTP, browser and connection evidence as separate results.

Distributed generators and controlled promotion stages

OPS.04 | OPERATIONS / LOAD-TEST EXECUTION

Use distributed load generators that are themselves capacity-tested. Grafana documents Kubernetes-based distributed execution through the k6 Operator, and its large-test guidance discusses generator constraints [OPS-S01, OPS-S02]. Pin the selected generator image and scripts. Reserve a separate environment or explicitly scoped test window; do not point an uncontrolled test at customer traffic.

Prevent the test from understating overload

Use arrival-rate scenarios for target request streams so server slowdown does not silently reduce offered traffic. Grafana describes this coupling in closed models as coordinated omission [OPS-S03]. Retain stateful session semantics separately; report scheduled requests, started requests, completed requests, dropped iterations and client scheduling delay. Do not treat a low error rate as success when the generator failed to produce the intended demand.

Benchmark each generator's CPU, memory, network, TLS cost, open connections and data-export overhead. Distribute actor ranges without overlaps or gaps; synchronize clocks and phases. Check source IP limits, egress bandwidth, provider quotas and internet routing. Reserve enough generator resources for the measured peak, and report whether traffic arrived through the public edge or a private network shortcut.

Promote six exact stages

Run 1,000 active actors for at least 15 minutes, then 10,000 for 30 minutes, 50,000 for 45 minutes, 100,000 for 60 minutes, 250,000 for 90 minutes and 500,000 for a full 180-minute steady-state soak after warmup. At each stage require the intended activity mix, valid receipt reconciliation, SLOs, bounded queue growth and no generator bottleneck. Exercise start and submission phases as separate recorded windows.

Promote only after the previous stage has a reviewed result. The final qualification must also include a full three-hour run with all 500,000 admitted attempts active; exclude ramp and preparation time from that proof. Keep cold-start, warm-cache and failure-injection runs distinct so a favorable warm run cannot conceal a failed cold or degraded scenario. Use scheduled-start fixtures to finish admissions and preflight before the common three-hour window; retain the start wave as a separately measured phase.

Aggregate pass/fail correctly

Use explicit k6 thresholds; ordinary checks alone do not define the complete release decision [OPS-S04]. Aggregate counts and merge compatible latency histograms across generators. Never average individual generators' p99 values into a global p99. Set delayed abort evaluation for noisy warmup only where documented, and keep immediate integrity-failure stops. Produce one signed run manifest, raw evidence references and global decision.

ACCEPTANCE / PRECAUTIONS

- Keep load generators outside production autoscaling pools.
- Fail under-generated or generator-saturated runs.
- Require operator approval before every larger stage.

Bursts, CDN fan-out and expiry storms

OPS.05 | OPERATIONS / START AND DEADLINE PRESSURE

Build explicit burst scenarios around the 120-second start wave and 30-second submission wave. Include the realistic payload distribution, authorization work, assets, database connections and receipt recovery. Add synchronized expiry and retry experiments. A healthy steady-state mean does not qualify the endpoints students need simultaneously at the beginning and end.

Exercise the whole start path

Test authentication token validation, session admission, cell routing, attempt creation, package-manifest retrieval and first-question rendering. For k initial asset requests per student, the start wave offers 500,000*k/120 requests/second to the content edge. Measure edge hit ratio, origin requests, origin-shield behavior, decompression and image decoding; do not assume CDN hits eliminate all origin work.

Use immutable versioned asset names and authorized access rules. Prewarm the intended question packages and test a cold or invalidated cache separately. Keep answer keys and embargoed solutions out of packages delivered before release. Verify the exact package and source-image hashes after fetch; a cached object from another paper version is an integrity failure, not merely a slow asset.

Remove synchronized renewal cliffs

Ensure authorized asset access lasts through the planned exam and reconnection allowance, or provide a tested renewal path before expiry. Stagger refresh with bounded jitter. Deliberately align signed-link expiry, token expiry and lease renewals in a fault run to reveal a renewal storm. A URL expiry must not revoke the student's committed response or change the timer.

Simulate temporary 429, 502, slow responses and lost acknowledgements. Apply bounded retry backoff and jitter, preserve idempotency keys and avoid retrying every failed endpoint at the same instant. Measure retry amplification = total attempts/original operations. Give commit and submission receipt recovery priority over optional telemetry, search, explanations and analytics refresh.

Exercise the last 30 seconds

Submit 500,000 attempts over 30 seconds with baseline activity overlapping. Include duplicate submits, network disconnect after persistence, worker backlog and deadline-triggered submissions. The submission endpoint must persist a receipt before acknowledging; grading may follow asynchronously. Verify one final accepted state and one grade effect per attempt/key revision. Queue lag may leave a pending grade, but it may not turn an acknowledged submission into an unanswered paper.

Check asset availability for students returning from background just before the deadline. Keep a server-confirmed submission state distinguishable from a locally queued request. A late-response decision belongs to the configured exam engine, with trusted ingress evidence preserved.

ACCEPTANCE / PRECAUTIONS

- Measure cold-cache and expiry-storm failures independently.
- Include retries when computing burst RPS.
- Verify durable receipt recovery after a lost client acknowledgement.

Fault injection and student-fair incident handling

OPS.06 | OPERATIONS / COVERED FAILURE ENVELOPE

Exercise failures against the qualified workload, not only an empty environment. Run isolated faults first and controlled combinations later. Record affected cells, precise timestamps, last trusted ingress watermark, accepted receipts, routing epochs and observed student states. An incident workflow must protect response truth before attempting to improve a latency dashboard.

Run the failure matrix

Inject an application process crash, a cell availability-zone loss, a database stall or failover, a cache outage, a queue-worker crash, duplicate and poison messages, and a telemetry outage. Add control-plane unavailability during already admitted sessions, a hot cell, source-asset failure and a bounded retry storm. Keep the control-plane outage test separate from the normal admission test.

For each fault define expected behavior: which requests can continue, which return retryable errors, which receipts remain recoverable and which reports become pending. Verify acknowledged committed answers survive the covered process, node and availability-zone failures. Do not promise zero regional data loss when the selected cross-region replication is asynchronous; the disaster scope and proposed RPO are addressed separately.

Prefer correct uncertainty over invented results

If persistence has not completed, the server cannot claim saved. If persistence completed but the acknowledgement was lost, an idempotent retry must return the original durable result. If trusted pre-deadline ingress evidence or a journal watermark is missing after failover, hold affected grading for reconciliation. Do not invent a receipt, backdate a response or substitute a local draft.

Quarantine poison messages after bounded attempts, with an auditable reason and safe replay path. Keep accepted attempt snapshots durable outside transient queues. Stop optional work before exhausting answer-write capacity. Do not silently drop already admitted attempts to meet a budget or restore a lower nominal load.

Apply fairness exceptions as explicit records

Use the platform's authorized operator PIN/step-up gate and scoped approval for a locked truth-policy exception. Record incident scope, reason, affected attempt set, approver, old policy, proposed adjustment and a new signed exception version. Do not rewrite historical deadlines or mutate the published policy beneath active attempts.

An extension, restart or result hold must follow the reviewed incident policy and remain distinct from answer-key corrections. Provide a cell-aware support diagnostic and an appeal record. Drafting an incident notice is permitted; transmitting email or external messages still requires the user's explicit authorization or an invoked authorized workflow.

ACCEPTANCE / PRECAUTIONS

- Reconcile every accepted response after each fault.
- Pause grading when trusted deadline evidence is uncertain.
- Keep historical timer policy and incident exceptions independently auditable.

Observability that scales without leaking attempt data

OPS.07 | OPERATIONS / METRICS, TRACES AND AUDIT

Implement an observation plan for each cell and the shared edge. Monitor offered load, accepted writes, durable acknowledgements, latency, database waits, queue age, collector drops and cohort freshness. Every alert must name an operator action. Preserve response correctness evidence in a separate protected audit ledger, not in an enormous metrics label set.

Use bounded metric dimensions

Allow approved low-cardinality dimensions such as cell, route template, outcome class, deployment digest family and adapter family. Reject learner ID, attempt ID, question ID, raw URL, email and idempotency token as metric labels. OpenTelemetry defines cardinality limits and overflow aggregation; configure limits and attribute filtering deliberately [OPS-S05]. Alert on overflow and unexpected time-series growth rather than letting monitoring degrade during the exam.

Store opaque attempt identifiers in access-controlled traces or audit records where needed for investigation. Do not attach answer payloads or personal notes to ordinary logs. Trace critical failures with bounded sampling and retain the durable receipt ledger independently of trace sampling. Sampling away a trace must never remove proof that an accepted answer was persisted.

Measure the actual service promises

Emit ingress-to-durable-ack histograms with consistent boundaries, endpoint outcome counters and accepted-commit reconciliation counts. Keep client round-trip and rendered-feedback measurements separately named. Under the submission burst measure durable receipt latency, receipt-to-core-grade latency and oldest pending grade age. Distinguish valid workload server errors, business-rule rejections, capacity denials and client cancellations; do not hide failures behind one success percentage.

Plot per-cell metrics as well as totals. Track database transaction duration, pool wait, storage latency, replication lag and failover epochs. Track queue depth with arrival and completion rates, not depth alone. Monitor admission capacity, asset-origin misses, expiry renewals and retries. Put separate limits on optional telemetry, analytics and search to preserve answer-write priority.

Verify alarms and retention

Inject threshold failures to prove alerts fire and the run evaluator sees them. Monitor the collectors themselves: export backlog, memory, rejected spans and missing metrics. A disabled or overloaded collector makes evidence incomplete, not a clean run. Establish retention for detailed audit data, metrics and traces, and verify access and deletion behavior.

Provide a support lookup by attempt ID that resolves the cell, lease and receipt state through authorized APIs. Return source-of-truth versions and a timestamp; avoid support screenshots becoming unofficial grading truth. Keep investigation actions and exports auditable.

ACCEPTANCE / PRECAUTIONS

- Reject high-cardinality labels in instrumentation review.
- Test alarms and collector-overload behavior.
- Keep sampled traces separate from complete durable receipt evidence.

Disaster recovery, backups and proof of restored truth

OPS.08 | OPERATIONS / RPO, RTO AND RESTORE VALIDATION

Define recovery point objective as allowable data loss and recovery time objective as restoration time, then test the actual selected deployment [OPS-S06]. Proposed targets are RPO zero for explicitly covered availability-zone failures using verified synchronous durable persistence, and regional-disaster RPO <= 5 minutes with RTO <= 60 minutes. These are acceptance targets, not vendor guarantees.

Disclose the regional loss boundary

Asynchronous cross-region replication can lose recently acknowledged data during unplanned failover; AWS explicitly describes this risk for Aurora Global Database [OPS-S07]. Measure actual replication lag and the last verified durable watermark. Do not restate the five-minute regional RPO as zero loss. A recovered response store must expose which attempts fall in a potentially missing interval.

Before promotion, establish one globally exclusive writer epoch or independently verified fencing of the old region's ingress and writer. If neither can be proved, remain unavailable rather than allow split-brain writes. Bind attempt routing and leases to the promoted epoch. Reconcile pre-deadline ingress ledgers and receipt hashes before final grading; missing evidence produces an explicit hold, not invented accepted answers.

Restore more than database rows

Back up the committed-response journal, attempt metadata, package manifests, approved keys, policy versions, incident exceptions, cohort eligibility snapshots and audit references. Protect independent backup vault access and recovery credentials. Validate encryption-key availability, KMS permissions, network configuration, database engine compatibility and immutable image/package access in the recovery environment. A backup status of completed does not prove decryptable or usable application recovery.

Use automated restore validation and application-level checks; AWS Backup provides restore-test validation workflows and requires relevant key access [OPS-S08, OPS-S09]. Restore to an isolated environment, verify package hashes, row counts, sequence bounds and receipt-to-response reconciliation, then rescore known fixtures. Check that rerunning jobs yields no duplicate score effects and no embargoed content exposure.

Measure the entire recovery exercise

Start the RTO clock at the first qualifying loss of essential exam service and stop only when authorized clients can use recovered essential APIs with validated data. Record detection, fencing, provisioning, restore, validation and routing durations separately. Measure RPO from acknowledged receipts against the recovered trusted watermark, not from a backup schedule.

Keep an operator recovery checklist with required approvals, rollback boundaries and affected attempts. Run restore and failover drills on the deployed configuration. If targets are missed, publish the measured result and block the capacity claim until remediation and another valid drill.

ACCEPTANCE / PRECAUTIONS

- Test KMS access and package integrity during restore.
- Prevent two regions accepting the same attempt.
- Disclose regional loss scope and reconcile before releasing grades.

Cost calculator, capacity reservations and safe budget controls

OPS.09 | OPERATIONS / MONEY AND RESOURCE GUARDRAILS

Build a transparent cost model with measured units and vendor prices supplied later. Do not invent an INR estimate or imply that the 500,000-student workload fits an unverified budget. Separate rehearsal cost, three-hour exam cost, retained data cost and recovery readiness. Budget controls must protect admitted exams rather than terminate students mid-session.

Calculate from the observed workload

Compute compute_cost = sum(instance_hours * quoted_hourly_price). Compute request_cost = sum(billable_requests/unit_size * quoted_unit_price), identifying load balancer, API, collector and storage-request categories without double counting. Compute transfer_cost = sum(billable_GB * quoted_price_per_GB) by edge, origin, inter-zone and inter-region category. Add database capacity, durable writes, backup storage, KMS, DNS, logs, metrics and generator resources.

Use actual request sizes, content package sizes, CDN miss ratios, compression, retention and retry amplification. Include 378 million baseline live operations and 180 million telemetry batches in the three-hour scenario, plus measured starts, submits and reads. Explain minimum billing increments and reserved versus on-demand capacity when vendor terms are supplied. Store currency, tax assumptions, quote date and region for every price.

Preapprove the test envelope

Before a large rehearsal, approve a maximum synthetic-test duration, generator fleet, target endpoints, expected request volume, egress allowance and spend ceiling. Alert early and abort the synthetic workload when its approved envelope is breached. Reserve cleanup tasks for test fixtures and resources. Keep load-test credentials and destination allowlists separate from student production credentials.

Check provider quotas for compute, load balancers, database connections, network egress, NAT and telemetry. Verify reserved headroom and cell placement before the start wave. A quota increase request or an autoscaling maximum is not proof of usable capacity. Recalculate the budget after each promotion stage from actual consumption, especially when retry amplification or collector cardinality increases.

Use admission controls rather than destructive cutoffs

Publish a per-cell capacity ledger and reserve the committed-write resources required by admitted sessions. Stop new admissions or optional workloads when protected margins are exhausted. Do not kill live attempts, revoke receipts, shrink persistence below the admitted envelope or force early submission to keep a cost chart green.

Apply priority limits to telemetry, analytics recomputation, admin exports and search. Explain how delayed optional features affect students. Operators must decide whether another exam cohort can be admitted based on qualified resources, current backlog and budget; that decision cannot be inferred solely from a low average CPU reading.

ACCEPTANCE / PRECAUTIONS

- Leave vendor unit prices TBD until quoted and verified.
- Include retries, backups and generator spend.
- Budget-stop synthetic tests while preserving admitted student sessions.

Feature availability and asynchronous analytical truth

OPS.10 | OPERATIONS / ALL 60 ANALYSES AND ADMIN FEATURES

Implement results as versioned asynchronous snapshots. The live attempt database must persist responses and receipts; it must not execute 60 raw-event queries whenever a student opens the report. Separate score readiness, cohort freshness, explanation release and evidence insufficiency. A delayed optional analysis must not block durable submission or corrupt the grade.

Map every analysis to its required evidence

A01-A15 require a finalized scoring ledger, approved taxonomy and relevant exposure history. A16-A30 require reconciled timing and commitment events, with the catalog's coverage gates. A31-A40 require matching eligible cohort snapshots, distinct first-unexposed learners and the catalog's sample and release gates. Keep platform cohort comparisons distinct from official NTA results.

A41-A45 require genuine optional precommit confidence or approved error labels, plus marking compatibility. A46-A55 require comparable longitudinal history, fresh checks or explicit prerequisite assignments. A56-A60 require solution-release and access evidence, old/new score revisions, approved mappings or timing-quality summaries. Return unavailable reasons when prerequisites are missing; do not manufacture a default value or infer medical conditions from timing.

Preaggregate cohorts without duplicate effects

Build exact score histograms using the adapter's finite scoring outcomes and exact mark units; never assume every exam uses integer marks. An inclusive platform percentile is 100*count(score <= learner_score)/N, preserving ties. At 500,000 eligible learners, update versioned counts incrementally rather than sorting or scanning the transactional table on each report request.

Key each cohort contribution by attempt, eligibility version and score revision. Apply removal of an old contribution and addition of a corrected contribution atomically. Preaggregate item delivered, attempted, correct, omitted and canonical option counts. Materialize timing distributions separately with clearly identified exact or approximate methods; an approximate speed summary must not masquerade as the catalog's exact inclusive time percentile. Publish a consistent cohort snapshot only after revision checks pass.

Keep admin and support views honest

Serve personal report snapshots through an authorized API with report revision, key version, cohort version and freshness. Batch report downloads and enforce explanation embargoes. Use a search index for discovery, then fetch source content by ID/version before editing or adjudicating. Display index lag; search absence is not proof that an approved item does not exist.

Provide cell-aware support tools for receipts, grading jobs, holds and incident exceptions. Apply role checks, cohort privacy suppression and audit logging. Optional index, report-export or analytics-worker failure should show pending status while the exam's response and deadline truth remain intact.

ACCEPTANCE / PRECAUTIONS

- Cover A01-A60 in the feature-state registry.
- Never query raw events per metric on the hot database.
- Verify corrected cohort snapshots remove stale contributions exactly once.

Thirteen sequential Emergent work packages (1/2)

OPS.11A | OPS.11 | OPERATIONS / IMPLEMENTATION HANDOFF

Emergent, implement these packages in order and attach their deliverables to one readiness dossier. The named endpoints are concrete contract examples: bind them to the project route manifest through explicit aliases where existing names differ. Internal endpoints must remain operator-authenticated and unavailable to students. Do not skip an exit gate because a UI demonstration looks complete.

Build contracts, traffic and correctness

- WP01: Deliver workload/SLO manifest; expose GET /internal/capacity/profile; test arithmetic and timing boundaries; exit only with approved failure scope and acceptance definitions.
- WP02: Deliver seeded actor fixtures; exercise POST /v1/attempts/start; test independent identities, adapters and canonical options; exit when expected ledgers reconcile for every fixture.
- WP03: Deliver durable commitment tests; exercise POST /v1/attempts/{id}/commit; inject lost acknowledgements, clears and duplicates; exit with zero accepted-response loss in covered faults.
- WP04: Deliver distributed generator manifests; expose authenticated run-status API; test actor partitioning, offered rates and scheduling delay; exit with verified generator capacity and no missing actor ranges.
- WP05: Deliver burst/expiry suite; exercise POST /v1/attempts/{id}/submit and package fetches; test overlap and receipt recovery; exit only when burst SLOs and integrity gates pass.

Evidence required at this checkpoint

Link each clause to its implementing module, API operation, storage constraint and a dated proof artifact. Retain the fixture input, expected result, measured output and operator conclusion. A diagram or screenshot demonstrates presentation but does not establish durable acceptance, permission enforcement, grading correctness or the full traffic envelope.

ACCEPTANCE / PRECAUTIONS

- Give each package an owner and evidence locations.
- Retain exact image digests and configuration versions.
- No deployment or external-message authorization is implied by this handoff.

Thirteen sequential Emergent work packages (2/2)

OPS.11B | OPS.11 | OPERATIONS / IMPLEMENTATION HANDOFF

Continue the same execution and production decision contract. Every requirement carries an owner and reproducible evidence; a failure or unqualified condition remains a release blocker.

Build operational safeguards and feature delivery

- WP06: Deliver nonproduction fault suite; expose internal fault-run status; test AZ, database, queue and control-plane outages; exit after all accepted receipts reconcile.
- WP07: Deliver dashboards and alarms; expose operator-only collector health; test cardinality overflow and alert firing; exit with complete metric evidence and bounded collection overhead.
- WP08: Deliver restore/failover runbook; expose GET /internal/recovery/status; test KMS, writer fencing and rescore fixtures; exit only with measured RPO/RTO and integrity evidence.
- WP09: Deliver cost/admission ledger; expose operator capacity status; test quota limits and synthetic budget stops; exit when admitted-session reservations survive optional-work shedding.
- WP10: Deliver A01-A60 feature registry; expose GET /v1/reports/{id}; test unavailable states, embargoes and cohort revisions; exit with snapshot/export parity and no hot-table analytical scans.
- WP11: Deliver admin/support diagnostics; expose versioned content and cell-aware receipt APIs; test stale indexes and permissions; exit when source versions and audit trails remain authoritative.

Release only immutable qualified artifacts

- WP12: Deliver IaC, signed images by digest and SBOM; expose authenticated release-manifest lookup; test backward-compatible migrations and rollback; exit without breaking active pinned attempts.
- WP13: Deliver staged run dossier and blocker register; expose internal readiness decision; test all six promotion stages; exit only with final three-hour evidence and named operator signoff.

ACCEPTANCE / PRECAUTIONS

- [ ] Give each package an owner and evidence locations.
- [ ] Retain exact image digests and configuration versions.
- [ ] No deployment or external-message authorization is implied by this handoff.

Production-blocking risk register and final decision (1/2)

OPS.12A | OPS.12 | OPERATIONS / PASS OR FAIL

Keep this register separate from the feature backlog. Give every blocker an owner, evidence and retest. Latency success cannot waive integrity or recovery gates. Mark untested requirements unqualified.

Capacity and correctness blockers

- OPS-R01: Pass only with 500,000 distinct active actor ledgers for 180 steady-state minutes; fail idle-user counts or incomplete activity mixes.
- OPS-R02: Pass only with intended offered rates and unsaturated generators; fail dropped scheduled traffic, private-path substitution or averaged local latency percentiles.
- OPS-R03: Pass only with zero incorrect grades, duplicate scoring effects or committed-A/draft-B violations; fail any unreconciled accepted-response payload.
- OPS-R04: Pass only when covered crash/AZ tests preserve acknowledged responses; fail receipts backed solely by volatile caches or transient queues.
- OPS-R05: Pass only when start, submit, cold-cache and expiry scenarios meet their recorded gates; fail a favorable steady average concealing critical-window failures.

Evidence required at this checkpoint

Link each clause to its implementing module, API operation, storage constraint and a dated proof artifact. Retain the fixture input, expected result, measured output and operator conclusion. A diagram or screenshot demonstrates presentation but does not establish durable acceptance, permission enforcement, grading correctness or the full traffic envelope.

ACCEPTANCE / PRECAUTIONS

- Block production claims while any critical gate is fail or unqualified.
- Retest after material configuration, schema or workload changes.
- Publish measured limitations beside the capacity decision.

Production-blocking risk register and final decision (2/2)

OPS.12B | OPS.12 | OPERATIONS / PASS OR FAIL

Continue the same execution and production decision contract. Every requirement carries an owner and reproducible evidence; a failure or unqualified condition remains a release blocker.

Recovery, feature and operating blockers

- OPS-R06: Pass only with tested writer fencing, usable keys and validated restore; fail split-brain promotion or unknown restored package/key versions.
- OPS-R07: Pass only with measured regional RPO/RTO and disclosed async-loss scope; fail unsupported zero-loss claims or grading beyond the trusted watermark.
- OPS-R08: Pass only with version-consistent report/cohort snapshots and embargo enforcement; fail stale scores, leaked solutions or duplicate cohort contributions.
- OPS-R09: Pass only with bounded metric labels, complete receipt audit and firing alarms; fail learner IDs in metrics or monitoring gaps labeled healthy.
- OPS-R10: Pass only with qualified cell reservations and approved synthetic-test budgets; fail cost controls that terminate admitted students or hide retry costs.
- OPS-R11: Pass only with signed digests, SBOM, reviewed IaC and compatible migrations; fail ungated production source edits or changes breaking active pinned attempts.
- OPS-R12: Pass only with audited incident exceptions and source-authoritative admin tools; fail silent deadline rewrites or stale indexes treated as grading truth.

Make the final decision reproducible

Attach the exact workload profile, deployment digests, cell configuration, measured rates, latency histograms, error windows, actor reconciliation, browser and connection results, failure drills, restoration evidence and cost inputs. Record each blocker as pass, fail or unqualified, with test identifiers. Obtain operator signoff on the declared capacity and failure envelope.

Promote a canary only after backward-compatible schema expansion, rehearsal and rollback validation. Keep attempt policies and content pinned for existing sessions; drain old leases before removing compatible fields or images. Halt new admissions on a failed gate, preserve already admitted sessions and retest the changed artifact. Do not silently carry approval from an earlier configuration into a new release.

ACCEPTANCE / PRECAUTIONS

- Block production claims while any critical gate is fail or unqualified.
- Retest after material configuration, schema or workload changes.
- Publish measured limitations beside the capacity decision.

Source register and implementation handoff (1/11)

EVIDENCE AND SCOPE OF THIS SPECIFICATION

Use these references to verify the stated baseline. Requirements and engineering thresholds in this volume are proposed product decisions unless a source explicitly establishes them.

MASTER-S01 | Emergent official deployment tutorial

https://emergent.sh/tutorials/how-to-deploy-your-app-on-emergent

Checked: 2026-10-12. Official description of managed deployment capabilities. It does not establish 500,000-concurrent examination capacity or verify this proposed external topology.

V-01 | AWS S3: Locking objects with Object Lock

https://docs.aws.amazon.com/AmazonS3/latest/userguide/object-lock.html

Checked: 2026-10-12. Primary documentation for object-version scope, compliance/governance retention, delete markers, and account-closure caveat. Operator-selected retention; no claim of infinite preservation.

V-02 | AWS S3: Object Lock considerations

https://docs.aws.amazon.com/AmazonS3/latest/userguide/object-lock-managing.html

Checked: 2026-10-12. Primary documentation for bypass governance permission, replication, delete markers, upload integrity, and encryption-key availability limitations.

V-03 | OWASP Password Storage Cheat Sheet

https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html

Checked: 2026-10-12. Primary guidance supporting Argon2id, unique salts, separate pepper storage, and target-system work-factor benchmarking. Exact deployment parameters remain operator-reviewed.

ACCEPTANCE / PRECAUTIONS

- Before release, retain the approved configuration, source document date, verification owner and test evidence. Re-check any rule that changes for a new exam year.

Source register and implementation handoff (2/11)

EVIDENCE AND SCOPE OF THIS SPECIFICATION

Use these references to verify the stated baseline. Requirements and engineering thresholds in this volume are proposed product decisions unless a source explicitly establishes them.

V-04 | OWASP Multifactor Authentication Cheat Sheet

https://cheatsheetseries.owasp.org/cheatsheets/Multifactor_Authentication_Cheat_Sheet.html

Checked: 2026-10-12. Primary guidance for actual MFA, risk-aware factor change and recovery, existing-factor reauthentication, and notification. PIN remains an additional knowledge gate rather than user identity or replacement MFA.

V-05 | AWS KMS API: Sign

https://docs.aws.amazon.com/kms/latest/APIReference/API_Sign.html

Checked: 2026-10-12. Primary signing API documentation. Signatures support verification against the corresponding public key; record algorithm, key identity, and canonical digest. No authorship or rights-ownership guarantee.

V-06 | OWASP Authorization Cheat Sheet

https://cheatsheetseries.owasp.org/cheatsheets/Authorization_Cheat_Sheet.html

Checked: 2026-10-12. Primary guidance for least privilege, deny by default, validating every request, and attribute/relationship-based controls.

V-07 | AWS KMS: Delete an AWS KMS key

https://docs.aws.amazon.com/kms/latest/developerguide/deleting-keys.html

Checked: 2026-10-12. Primary documentation for irreversible deletion risks, pending-deletion unavailability, cancellation before deletion, and account-closure dependency. Avoids universal recovery or export claims.

ACCEPTANCE / PRECAUTIONS

- Before release, retain the approved configuration, source document date, verification owner and test evidence. Re-check any rule that changes for a new exam year.

Source register and implementation handoff (3/11)

EVIDENCE AND SCOPE OF THIS SPECIFICATION

Use these references to verify the stated baseline. Requirements and engineering thresholds in this volume are proposed product decisions unless a source explicitly establishes them.

V-08 | OWASP Logging Cheat Sheet

https://cheatsheetseries.owasp.org/cheatsheets/Logging_Cheat_Sheet.html

Checked: 2026-10-12. Primary guidance for logging decisions, excluding credentials and sensitive tokens, sanitizing event data, tamper detection, and restricting audit access.

V-09 | OWASP File Upload Cheat Sheet

https://cheatsheetseries.owasp.org/cheatsheets/File_Upload_Cheat_Sheet.html

Checked: 2026-10-12. Primary guidance for type validation, private storage, antivirus/sandbox checks, resource controls, and content-disarm derivatives.

V-10 | OWASP Server Side Request Forgery Prevention Cheat Sheet

https://cheatsheetseries.owasp.org/cheatsheets/Server_Side_Request_Forgery_Prevention_Cheat_Sheet.html

Checked: 2026-10-12. Primary guidance for constrained server-side fetching and private/network metadata endpoint protection. Parser isolation and no-network policy are proposed implementation requirements.

V-11 | AWS CloudTrail: Validating log file integrity

https://docs.aws.amazon.com/awscloudtrail/latest/userguide/cloudtrail-log-file-validation-intro.html

Checked: 2026-10-12. Primary documentation for signed digest files and the requirement to perform actual validation after enabling integrity-digest delivery.

ACCEPTANCE / PRECAUTIONS

- Before release, retain the approved configuration, source document date, verification owner and test evidence. Re-check any rule that changes for a new exam year.

Source register and implementation handoff (4/11)

EVIDENCE AND SCOPE OF THIS SPECIFICATION

Use these references to verify the stated baseline. Requirements and engineering thresholds in this volume are proposed product decisions unless a source explicitly establishes them.

V-12 | AWS CloudFront: Restrict access to an Amazon S3 origin

https://docs.aws.amazon.com/AmazonCloudFront/latest/DeveloperGuide/private-content-restricting-access-to-s3.html

Checked: 2026-10-12. Primary documentation recommends OAC and confirms OAI lacks SSE-KMS support. Origin access controls do not substitute for viewer authorization before cache hits. The pre-cache viewer gate is the proposed product security contract.

API-S01 | OpenAPI Specification v3.1.1

https://spec.openapis.org/oas/v3.1.1.html

Checked: 2026-10-12. Primary pinned interface specification. This is an intentional 3.1 reference, not a claim that it is the latest OpenAPI release.

API-S02 | JSON Schema Draft 2020-12

https://json-schema.org/draft/2020-12

Checked: 2026-10-12. Primary schema vocabulary and dialect documentation. Ownership, timing and academic rules remain application checks.

API-S03 | RFC 9110: HTTP Semantics

https://www.rfc-editor.org/rfc/rfc9110.html

Checked: 2026-10-12. Primary standard for method, status, header and conditional-request semantics. Platform receipt retention and retry rules are explicit design requirements.

ACCEPTANCE / PRECAUTIONS

- Before release, retain the approved configuration, source document date, verification owner and test evidence. Re-check any rule that changes for a new exam year.

Source register and implementation handoff (5/11)

EVIDENCE AND SCOPE OF THIS SPECIFICATION

Use these references to verify the stated baseline. Requirements and engineering thresholds in this volume are proposed product decisions unless a source explicitly establishes them.

API-S04 | OWASP API1:2023 Broken Object Level Authorization

https://api-security.owasp.org/editions/2023/en/0xa1-broken-object-level-authorization/

Checked: 2026-10-12. Primary security guidance supporting object ownership checks on referenced resources.

API-S05 | OWASP API3:2023 Broken Object Property Level Authorization

https://api-security.owasp.org/editions/2023/en/0xa3-broken-object-property-level-authorization/

Checked: 2026-10-12. Primary security guidance supporting allowlisted response properties and protected mutation fields.

API-S06 | Stripe API: Idempotent requests

https://docs.stripe.com/api/idempotent_requests

Checked: 2026-10-12. Primary provider example of explicit idempotency-key behavior. This platform uses its own documented scope, receipt retention and business uniqueness protections.

API-S07 | Stripe documentation: Webhooks

https://docs.stripe.com/webhooks

Checked: 2026-10-12. Primary provider guidance for raw-body signature verification, duplicates and asynchronous handling; optional payments require the selected provider contract.

ACCEPTANCE / PRECAUTIONS

- Before release, retain the approved configuration, source document date, verification owner and test evidence. Re-check any rule that changes for a new exam year.

Source register and implementation handoff (6/11)

EVIDENCE AND SCOPE OF THIS SPECIFICATION

Use these references to verify the stated baseline. Requirements and engineering thresholds in this volume are proposed product decisions unless a source explicitly establishes them.

API-S08 | PostgreSQL 18: Date/Time Functions, Section 9.9.5

https://www.postgresql.org/docs/current/functions-datetime.html

Checked: 2026-10-12. Primary database documentation: clock_timestamp() samples actual current time; now() and CURRENT_TIMESTAMP reflect transaction start. Sampling after the attempt lock and durable deadline registration are platform protocol requirements.

SCALE-S01 | AWS Well-Architected: What is a cell-based architecture?

https://docs.aws.amazon.com/wellarchitected/latest/reducing-scope-of-impact-with-cell-based-architecture/what-is-a-cell-based-architecture.html

Checked: 2026-10-12. Primary reference for independent cells, thin routing and separate control plane. The 20 x 25,000 design is proposed application sizing.

SCALE-S02 | Kubernetes official documentation: Horizontal Pod Autoscaling

https://kubernetes.io/docs/concepts/workloads/autoscaling/horizontal-pod-autoscale/

Checked: 2026-10-12. Primary reference for resource/custom metrics and HPA behavior; application capacity and timing require measurement.

SCALE-S03 | PostgreSQL official documentation: Table Partitioning

https://www.postgresql.org/docs/current/ddl-partitioning.html

Checked: 2026-10-12. Primary reference for declarative partitioning and constraint implications. Schema, write amplification and capacity model are proposed.

ACCEPTANCE / PRECAUTIONS

- Before release, retain the approved configuration, source document date, verification owner and test evidence. Re-check any rule that changes for a new exam year.

Source register and implementation handoff (7/11)

EVIDENCE AND SCOPE OF THIS SPECIFICATION

Use these references to verify the stated baseline. Requirements and engineering thresholds in this volume are proposed product decisions unless a source explicitly establishes them.

SCALE-S04 | Amazon RDS official documentation: Multi-AZ DB instance deployments

https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/Concepts.MultiAZSingleStandby.html

Checked: 2026-10-12. Primary reference for synchronous standby deployment and increased commit latency. Do not confuse with Multi-AZ DB cluster semantics.

SCALE-S05 | PostgreSQL official documentation: Log-Shipping Standby Servers

https://www.postgresql.org/docs/current/warm-standby.html

Checked: 2026-10-12. Primary reference for asynchronous replication loss window and synchronous replication options. Actual provider policy and fault tests govern guarantees.

SCALE-S06 | Amazon CloudFront official documentation: Quotas

https://docs.aws.amazon.com/AmazonCloudFront/latest/DeveloperGuide/cloudfront-limits.html

Checked: 2026-10-12. Lists 150 Gbps and 250,000 requests per second default per distribution with flat-rate-plan exceptions. Defaults are not reserved or tested application capacity.

SCALE-S07 | Elastic Load Balancing official documentation: ALB quotas

https://docs.aws.amazon.com/elasticloadbalancing/latest/application/load-balancer-limits.html

Checked: 2026-10-12. Primary reference for regional/account quotas, targets, rules and LCU reservations. Inspect actual account and approval state.

ACCEPTANCE / PRECAUTIONS

- Before release, retain the approved configuration, source document date, verification owner and test evidence. Re-check any rule that changes for a new exam year.

Source register and implementation handoff (8/11)

EVIDENCE AND SCOPE OF THIS SPECIFICATION

Use these references to verify the stated baseline. Requirements and engineering thresholds in this volume are proposed product decisions unless a source explicitly establishes them.

SCALE-S08 | Amazon S3 official documentation: Optimizing performance

https://docs.aws.amazon.com/AmazonS3/latest/userguide/optimizing-performance.html

Checked: 2026-10-12. Primary reference for prefix request rates, gradual scaling, 503 Slow Down and KMS limits. No object-store benchmark is claimed.

SCALE-S09 | AWS WAF official documentation: Quotas

https://docs.aws.amazon.com/waf/latest/developerguide/limits.html

Checked: 2026-10-12. Primary reference for 100,000 requests per second per regional protection pack/web ACL and separate CloudFront scope. Verify current actual quota arrangement before launch.

SCALE-S10 | Amazon RDS official documentation: Multi-AZ DB cluster deployments

https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/multi-az-db-clusters-concepts.html

Checked: 2026-10-12. Primary reference describing the semisynchronous Multi-AZ DB cluster offering, distinct from synchronous single-standby DB instance deployment.

SCALE-S11 | Elastic Load Balancing official documentation: Application Load Balancer introduction

https://docs.aws.amazon.com/elasticloadbalancing/latest/application/introduction.html

Checked: 2026-10-12. Primary reference for HTTP application-layer routing and target health. No benchmark or application authorization guarantee is derived from vendor scaling descriptions.

ACCEPTANCE / PRECAUTIONS

- Before release, retain the approved configuration, source document date, verification owner and test evidence. Re-check any rule that changes for a new exam year.

Source register and implementation handoff (9/11)

EVIDENCE AND SCOPE OF THIS SPECIFICATION

Use these references to verify the stated baseline. Requirements and engineering thresholds in this volume are proposed product decisions unless a source explicitly establishes them.

SCALE-S12 | Elastic Load Balancing official documentation: Network Load Balancer introduction

https://docs.aws.amazon.com/elasticloadbalancing/latest/network/introduction.html

Checked: 2026-10-12. Primary reference for transport-layer load distribution and connection behavior. Stable attempt ownership must still be enforced by the application.

SCALE-S13 | PostgreSQL official documentation: Date/Time Functions, section 9.9.5

https://www.postgresql.org/docs/current/functions-datetime.html

Checked: 2026-10-12. CURRENT_TIMESTAMP and now() use transaction-start time; clock_timestamp() reads actual current time. Deadline authority must read the fresh clock after serialized admission lock acquisition.

OPS-S01 | Grafana k6: Running distributed tests

https://grafana.com/docs/k6/latest/testing-guides/running-distributed-tests/

Checked: 2026-10-12. Primary documentation for distributed execution using k6 Operator. Tool capability is distinct from a qualified 500,000-session application capacity claim.

OPS-S02 | Grafana k6: Running large tests

https://grafana.com/docs/k6/latest/testing-guides/running-large-tests/

Checked: 2026-10-12. Primary generator-capacity and large-test guidance. Pin tool versions and validate the selected distributed aggregation path.

ACCEPTANCE / PRECAUTIONS

- Before release, retain the approved configuration, source document date, verification owner and test evidence. Re-check any rule that changes for a new exam year.

Source register and implementation handoff (10/11)

EVIDENCE AND SCOPE OF THIS SPECIFICATION

Use these references to verify the stated baseline. Requirements and engineering thresholds in this volume are proposed product decisions unless a source explicitly establishes them.

OPS-S03 | Grafana k6: Open and closed models

https://grafana.com/docs/k6/latest/using-k6/scenarios/concepts/open-vs-closed/

Checked: 2026-10-12. Primary explanation of arrival-rate scheduling and coordinated omission. Workload rates and session state in this section are project proposals.

OPS-S04 | Grafana k6: Thresholds

https://grafana.com/docs/k6/latest/using-k6/thresholds/

Checked: 2026-10-12. Primary threshold and delayed-abort documentation. Also verify https://grafana.com/docs/k6/latest/using-k6/checks/; checks alone do not make the entire test a failed run.

OPS-S05 | OpenTelemetry Metrics SDK: Cardinality limits

https://opentelemetry.io/docs/specs/otel/metrics/sdk/

Checked: 2026-10-12. Primary specification for cardinality limits, attribute filtering and overflow aggregation; validate language SDK support and exporter behavior.

OPS-S06 | AWS Well-Architected: REL 13 disaster recovery

https://docs.aws.amazon.com/wellarchitected/latest/framework/rel-13.html

Checked: 2026-10-12. Primary recovery-objective framework. RPO/RTO targets here are proposed acceptance criteria and must be tested against the selected deployment.

ACCEPTANCE / PRECAUTIONS

- Before release, retain the approved configuration, source document date, verification owner and test evidence. Re-check any rule that changes for a new exam year.

Source register and implementation handoff (11/11)

EVIDENCE AND SCOPE OF THIS SPECIFICATION

Use these references to verify the stated baseline. Requirements and engineering thresholds in this volume are proposed product decisions unless a source explicitly establishes them.

OPS-S07 | Amazon Aurora: Global Database switchover and failover

https://docs.aws.amazon.com/AmazonRDS/latest/AuroraUserGuide/aurora-global-database-disaster-recovery.html

Checked: 2026-10-12. Primary statement that unplanned cross-region failover can lose data due to asynchronous replication lag. Applies specifically to this database variant; do not generalize vendor guarantees.

OPS-S08 | AWS Backup: Restore testing validation

https://docs.aws.amazon.com/aws-backup/latest/devguide/restore-testing-validation.html

Checked: 2026-10-12. Primary restore-validation workflow. Application receipt, grading, policy and package-hash checks in this section are project-required additions.

OPS-S09 | AWS Backup: Restore testing

https://docs.aws.amazon.com/aws-backup/latest/devguide/restore-testing.html

Checked: 2026-10-12. Primary restore-test and key-access documentation. A completed backup is not sufficient evidence for recovery of the complete exam system.

ACCEPTANCE / PRECAUTIONS

- Before release, retain the approved configuration, source document date, verification owner and test evidence. Re-check any rule that changes for a new exam year.